

claude-windows / 78dabe12-41fd-46c2-ae90-ab74bdaec3c6

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\78dabe12-41fd-46c2-ae90-ab74bdaec3c6\subagents\workflows\wf\_18dcd7ff-5ae\agent-a18a923ac2fdaf967.jsonl`
- SHA-256: `f13424c286c895615b942a8356f4d8f7c6a90b5dc4a51e8ad43086733488be4f`
- Source modified: `2026-06-22T10:27:54+00:00`
- Imported at: `2026-07-05T16:48:26+00:00`
- project: `wf\_18dcd7ff-5ae`
- session_id: `78dabe12-41fd-46c2-ae90-ab74bdaec3c6`

Transcript

1. user (2026-06-22T10:25:19.417Z)

CONTEXT ? a GCS corpus-migration work item just completed on branch work/golden-corpus-hygiene (PR #316, repo Khelsutra/badminton-highlight-indexer). Goal: run ALL training+inference from the cloud with ALL data in gs://khelsutra-rally-corpus so the local Windows box + external drives stop being blockers.

AUTHORITATIVE DOC TO REVIEW: docs/DATA_IN_GCS.md (the corpus source-of-truth map).
RELEVANT CODE: backend/worker/__main__.py (cmd_infer ~157, cmd_train ~332, --corpus-uri ~473), backend/storage/ref.py (URI schemes/resolve_input_uri), deploy/gcp/README.md, docs/SETUP_NEW_MACHINE.md, docs/archives/past_projects/DECOUPLED_COMPUTE/.

VERIFIED BUCKET STATE (already confirmed deterministically this session ? treat as ground truth, do NOT

re-run gcloud, it may not be authed for you):

- trajectories/ = 15 objects (<name>.csv)
- labels/ = 15 canonical objects only, named <YYYY-MM-DD>_<name>.rallies.csv (date = earliest mtime across C:/F: copies); the 7 old inconsistent (*_proxy / bare GX010128,mahadevpura-1) were MD5-verified byte-identical dupes and DELETED.
- manifests/golden.json = 15 entries, every trajectory+label gs:// ref RESOLVES (0 dangling).
- videos/ = mahadevpura_1.mp4 (523MB, demo of Drive->bucket) + 3 pre-existing (GX030094_proxy.mp4, mahadevpura-1.MP4, mahadevpura_smoke_180s.mp4). 14 of 15 golden videos NOT yet uploaded.
- worker train reads labels/(+videos/) and BUILDS the manifest on-box (does NOT read manifests/golden.json); manifests/golden.json is for EVAL. trajectories are a cache (training can regenerate via --trajectory-source detector).
- Open: Phase-2 videos (7 Drive proxies + 8 large F: full-MP4s, proxy-first), Phase-3 flip eval/litmus to GCS.
- Sibling threads (from memory): a golden-fixtures P3 thread needs these same golden traj+label CSVs; the served Gate-A litmus (#236, merged) is pinned to local paths; original-6 labels turned out recoverable from F:.

LENS = COMPLETENESS CRITIC. The goal is "run ALL train+infer from the cloud, remove the local box as a blocker." Read the doc + SETUP_NEW_MACHINE.md + deploy/gcp/README.md. What is MISSING to actually reach that goal that the doc/migration does not yet cover or flag? (e.g. SA/creds on the cloud box, GPU quota, secrets, the 14 un-uploaded videos, proxy generation for the 8 large originals, CI/eval still reading local paths, the local litmus not flipped, who runs Phase-2/3). Produce the gap list a next session would hit.

Return ONLY the structured verdict.

2. assistant (2026-06-22T10:25:22.888Z)

I'll review the migration for completeness against the goal of running all train+infer from the cloud. Let me start by reading the authoritative doc and related files.

3. user (2026-06-22T10:25:24.529Z)

```
53467c9 docs(data): record stale-label cleanup + set up Phase-2 video copy
6ae3503 docs(data): record Phase-1 GCS migration (trajectories + labels + manifest)
8884c9f docs(data): add DATA_IN_GCS.md ? the corpus source-of-truth map (GCS)
689c738 docs(pkg): mark P2c ? + the loop closed ? v1 LIGHT is feature-complete (#313)
293e5c0 feat(pkg): re-bundle backend/ui ? serve the P2c first-run wizard from indexer --app
(#312)
---BRANCH---
work/golden-corpus-hygiene
---STATUS---
?? artifacts/
?? docs/KHELSUTRA_PRODUCT_OVERVIEW.md
```

4. assistant (2026-06-22T10:25:27.382Z)

Let me read the key files in parallel.

5. user (2026-06-22T10:25:28.343Z)

```
1      # Data in GCS ? the corpus source-of-truth map
2
3      **Status:** LIVE / in-progress (migration not complete). **Owner-gated** decisions in
$7.
4      **Last inventoried:** 2026-06-22 against `gs://khelsutra-rally-corpus`.
5      **Goal (owner, 2026-06-22):** run **all** training + inference from the cloud with **all
data in GCS**,
6      so the local Windows box (and the intermittently-mounted external drive) is **no longer
a blocker**.
7
8      > **? Other sessions: this is where the data lives.** Pull from `gs://khelsutra-rally-
corpus`, not
9      > from anyone's `C:` / `F:` / Google Drive. If something you need isn't in the bucket
yet, it's in the
10     > **gap table ($5)** ? flag it / migrate it, don't hard-code a local path.
11
12     This doc is the **data map**. The *machine* runbooks are
[ `SETUP_NEW_MACHINE.md` ](SETUP_NEW_MACHINE.md)
13     (spin up a box) and [ `deploy/gcp/README.md` ](.. / deploy/gcp/README.md) (provision GPU +
train/infer).
14     The cloud engine itself was built by the (archived) DECOUPLED_COMPUTE project
15     ( `docs/archives/past_projects/DECOUPLED_COMPUTE/` ); this doc does **not** redesign it ?
it records
16     *where the bytes are* and *what's still missing*.
17
18     ---
19
20     ## 1. The bucket
21
22     | | |
23     |---|---|
24     | Bucket | **`gs://khelsutra-rally-corpus`** (single corpus bucket) |
25     | Project | `gen-lang-client-0306026826` (billing *Khelsutra Billing*) |
26     | Region | `asia-south1` (Mumbai), UBLA, co-located with the GPU VMs |
27     | Auth | SA `rally-worker@gen-lang-client-0306026826.iam.gserviceaccount.com`
( `roles/storage.objectAdmin` ); key at `~/gcp/rally-worker-sa.json` (local only, **never
committed**). Interactive: `gcloud auth login`. |
```

```

28 | Size today | ~2.06 GB (mostly past dry-run reels/outputs ? **not** the full corpus) |
29
30 The engine is **backend-agnostic by URI** (`backend/storage/ref.py`): `gcs://` (or
31 `gs://`),
32 `s3://`, `gdrive://`, or a bare local path. Pick the backend with **config + creds, no
33 code change**.
34 ---
35 ## 2. Canonical layout (the target contract)
36
37 The cloud worker's input/output contract (`backend/worker/__main__.py`; lineage:
38 `DECOUPLED_COMPUTE/04`):
39
40 | Prefix | Holds | Who reads/writes it |
41 |---|---|---|
42 | `videos/` | source videos / proxies (`<clip>.mp4`) | **input** to `worker infer` &
43 `worker train` |
44 | `labels/` | golden rally labels (`<clip>.rallies.csv`) | **input** ground truth to
45 `worker train` + all eval |
46 | `trajectories/` | cached WASB trajectory CSVs | ** (PROPOSED ? see §7.2) ** GPU-free
47 eval/litmus input |
48 | `weights/<version>/` | trained Gen-0 bundles (`weights.json` + a run `manifest.json`)
49 | **output** of `worker train` |
50 | `models/` | upstream pretrained weights (`wasb_badminton_best.pth.tar`) | input to the
51 WASB detector |
52 | `outputs/<video_id>/` | per-video inference outputs (intervals, etc.) | **output** of
53 `worker infer` |
54 | `highlights/`, `promo/` | rendered reels + promo artifacts | output / demo |
55
56 > **Note on the manifest.** `worker train` does **not** read a pre-staged corpus
57 `manifest.json` ? it
58 > *builds* one on-box at train time from `labels/` (+`videos/`), extracting trajectories
59 via the native
60 > detector (`--trajectory-source detector`). A `manifest.json` only appears as an
61 **output** inside each
62 > `weights/<version>/` bundle. The **eval/litmus** path is what wants a stable, path-
63 resolvable manifest
64 > (§7.3).
65
66 ---
67
68 ## 3. What's actually in the bucket today (2026-06-22)
69
70 > **Phase-1 landed 2026-06-22:** added `trajectories/` (15), 15 canonical
71 `labels/<date>_<name>.rallies.csv`,
72 > and `manifests/golden.json`. The table below is the **pre-Phase-1** snapshot + the
73 *remaining* gaps
74 > (videos, the 7 stale label objects to delete) ? see §7/§8.
75
76 | Prefix | Present | Drift / notes |
77 |---|---|---|
78 | `labels/` | 7 files: `GX010128`, `mahadevpura-1`, + `Badminton BXH 2_proxy`, `Boxhill
79 Doubles_proxy`, `GX020094_proxy`, `GX030094_proxy`, `mahadevpura-2_proxy` | ? **inconsistent
80 naming** (`_proxy` on some, not others); ? **missing** the original-6 labels + `GX010137`,
81 `GX010141` |
82 | `videos/` | 3: `GX030094_proxy.mp4`, `mahadevpura-1.MP4`, `mahadevpura_smoke_180s.mp4`
83 | ? corpus videos/proxies almost entirely absent |
84 | `trajectories/` | ? | ? **does not exist** (every trajectory is local-only) |
85 | `models/` | `wasb_badminton_best.pth.tar` | ? (? WASB-C3 weight-provenance still an
86 open legal item before any external share) |
87 | `weights/` | `0.1.0-l4/` | ? first real L4 train bundle |
88 | `outputs/` | one `<video_id>/`, `parity/` | ? dry-run outputs |
89 | `highlights/`, `promo/` | reels + promo | ? |

```

```

74     **Drift summary:** the bucket has extra prefixes (`highlights/`, `promo/`, `models/`
alongside
75     `weights/`) and **inconsistent `labels/` naming**, and is **missing the bulk of the
corpus inputs**
76     (`videos/`, all `trajectories/`, most `labels/`). It reflects past *dry runs*, not a
complete corpus.
77
78     ---
79
80     ## 4. How the cloud consumes it
81
82     Provisioning + the verified commands live in
[ `deploy/gcp/README.md` ](../deploy/gcp/README.md) $3?$5.
83     The data-facing entry points (`backend/worker/__main__.py`):
84
85     ```bash
86     # config (no secrets in config; auth via GOOGLE_APPLICATION_CREDENTIALS):
87     #   { "storage": { "backend": "local", "gcs": { "project": "gen-lang-client-0306026826"
} } }
88
89     # INFERENCE on one cloud video -> outputs/<video_id>/ in the bucket
90     python -m backend.worker infer \
91         --video-uri gcs://khelsutra-rally-corpus/videos/<clip>.mp4 \
92         --out-uri gcs://khelsutra-rally-corpus --segmenter wasb_hybrid --config config.json
93
94     # TRAIN from the bucket (reads labels/ [+ videos/], extracts trajectories, builds
manifest on-box,
95     # runs training.gen0.harness, pushes weights/<version>/):
96     RALLY_DETECTOR_IMPL=native python -m backend.worker train \
97         --corpus-uri gcs://khelsutra-rally-corpus --out-uri gcs://khelsutra-rally-corpus \
98         --version <v> --trajectory-source detector --segmenter wasb_hybrid --config
config.json
99     ```
100
101     A bare/relative path resolves under a configured `input_root`/`input_backend`
102     (`backend/storage/ref.py::resolve_input_uri`), so workflows can be written bucket-
relative.
103
104     ---
105
106     ## 5. The gap ? what's still local-only (why the box is still a blocker)
107
108     All of this is **tiny except the originals** ? the metadata that blocks cloud eval is <
25 MB total:
109
110     | Data | Where it is now | Size | In bucket? | Needed for |
111     |---|---|---|---|---|
112     | 20 WASB trajectory CSVs | `C:\?\Annotation Setup\Collect\Trajectories\` | **17.5 MB**
| ? none | eval/litmus (cached), train (`--trajectory-source cached`) |
113     | golden manifest | `output/human_lovo_manifest.json` (C:-absolute paths) | 4.5 KB | ? |
eval corpus definition |
114     | original-6 labels | `F:\[Khelsutra]\Training\Golden Labelled\*.rallies.csv` | ~12 KB
| ? | train + eval |
115     | 7 new-clip labels | repo `output/*.rallies.csv` (gitignored) **** F: archive (backed
up 2026-06-22) | ~12 KB | ? partial/inconsistent | train + eval |
116     | source videos / proxies | F: archive + Google Drive golden folder | **100s of GB** | ?
3 only | infer + train (real WASB extraction) |
117
118     > The corpus *metadata* (trajectories + labels + manifest, ~25 MB) is what actually
blocks GPU-free
119     > cloud eval and the served Gate-A litmus ? and it's trivially small to migrate. The
**videos** are the
120     > only large migration, and they already exist on **Google Drive** (the golden-corpus
folder), so they
121     > can go **Drive ? bucket from a cloud box**, keeping the local machine out of the path

```

```

entirely.
122
123    ---
124
125    ## 6. Migration plan (close the gap)
126
127    **Phase 1 ? corpus metadata (?25 MB, do first, removes the eval blocker).** Stage all
trajectories,
128    all labels, and a **bucket-relative eval manifest** into `gs://khelsutra-rally-corpus`.
Pending the $7
129    naming decision. Sketch:
130
131    ```bash
132    # trajectories (decision-free; absent today)
133    gcloud storage cp "C:\?\Collect\Trajectories\*.csv" gs://khelsutra-rally-
corpus/trajectories/
134    # labels (after the $7.1 naming decision) -> labels/<canonical-name>.rallies.csv
135    # eval manifest with gs:// (or bucket-relative) paths -> a stable manifest object ($7.3)
136    ```
137
138    **Phase 2 ? videos ? `videos/<name>.mp4`** (bucket name = the eval `name`, consistent
with
139    `trajectories/` + `labels/`). Sources (resolved 2026-06-22 ? see
`scratch/copy_videos_to_gcs.ps1`):
140
141    | Source | Clips | Size |
142    |---|---|---|
143    | Drive golden folder `~/Golden Label Videos Request - June 15/[Done] Video N/`
(**proxies**) | `mahadevpura_1/2`, `GX010128`, `GX020094`, `GX030094`, `Badminton_BXH_2`,
`Boxhill_Doubles` (7) | ~13.5 GB |
144    | F: archive (**full MP4s** ? proxy-first before upload) | `GX010137`, `GX010141`, +
original-6 (`adarsh_avi_singles`, `kushagra_singles`, `largetest_doubles`, `gbaaddy`,
`testlarge_short`, `mahadevpura_singles`) (8) | ~50 GB |
145
146    **Two ways to run it:**
147    - **Decoupled (preferred ? no local box):** `rclone` on a cloud VM with a Google-Drive
remote ?
148    `rclone copy gdrive:"<?/[Done] Video N>/<proxy>.mp4" :gcs:khelsutra-rally-
corpus/videos/<name>.mp4`
149    (one-time owner step: `rclone config` ? `gdrive` remote OAuth). Or `gdown <file-id>`
on the box for
150    the shared folder. This is the path that actually removes the local box.
151    - **Interim (today, owner box ? streams Drive/F: ? local ? GCS):** `pwsh
scratch/copy_videos_to_gcs.ps1
152    -Execute` (dry-run without `-Execute`; `-Only <name>` for one; skip-existing). Uses
local bandwidth.
153
154    **Priority:** the 7 Drive **proxies** first (already 1080p, sufficient for detection).
The 8 large F:
155    full MP4s should be **proxied to 1080p first** (cheaper storage + faster cloud decode)
before upload.
156    ? Two pre-existing non-canonical video objects (`videos/GX030094_proxy.mp4`,
`videos/mahadevpura-1.MP4`)
157    are superseded once `videos/GX030094.mp4` / `videos/mahadevpura_1.mp4` land ? verify-
dupe-then-delete
158    (same as the label cleanup). Track per-clip in $8.
159
160    **Phase 3 ? flip eval/training to read GCS by default.** Point the eval manifest +
litmus at the
161    bucket (env override or `input_root`) so a fresh box/CI needs no local corpus. Re-verify
the suite +
162    the served Gate-A litmus read from `gcs://` (and re-measure if the corpus set changes ?
see PR #236).
163
164    ---

```

```

165
166     ## 7. Decisions
167
168     1. **`labels/` naming ? LOCKED (owner, 2026-06-22):** `labels/<YYYY-MM-DD>_<name>.rallies.csv`, where
169         `<name>` is the eval/manifest clip name and `<YYYY-MM-DD>` is the label's **authoring date** =
170         the *earliest* mtime across all known copies (C: working copy + F: archive ? robust to copy-resets;
171         a restored C: copy shows today, the F: archive keeps the real date). Fixes the
172         `_proxy` inconsistency
173         and disambiguates recycled GoPro names by date. *(Future, when videos are in the bucket: optionally
174         add a `video_id`/MD5 sidecar for the strongest keying ? deferred, needs hashing each video.)*
175     2. **Cache `trajectories/` ? YES, done 2026-06-22.** `trajectories/<name>.csv` cached so cloud/CI eval +
176         the served Gate-A litmus run **GPU-free** (training can still regenerate via `--trajectory-source detector`).
177     3. **Eval manifest in GCS ? done 2026-06-22.** `gs://khelsutra-rally-corpus/manifests/golden.json`
178         (15 clips, `gs://` trajectory + label paths). The **local** litmus keeps its local-path manifest;
179         flipping the litmus to read the GCS manifest is Phase 3.
180     4. **Originals migration scope ? OPEN** ? proxies-only vs full 4K, and how much of the 15-clip golden
181         set vs the wider archive. Source = Google Drive ? bucket from a cloud box (not the local box).
182
183     > Phase-1 was run via a dry-run planner (resolve every clip's trajectory + label, stamp the label's
184     > authoring date, build the `gs://` manifest) ? `gcloud storage cp` (planner kept in gitignored
185     > `scratch/`; promote it to `scripts/` if corpus-sync recurs).
186     > ? **Cleanup DONE 2026-06-22:** the 7 pre-existing inconsistent label objects (`_proxy.rallies.csv`,
187     > bare `GX010128`/`mahadevpura-1`) were confirmed **byte-identical (MD5)** to the new canonical names
188     > (content also on C:/F:) and **deleted** ? `labels/` now holds exactly the 15 canonical objects.
189
190     > ? **gcloud gotcha (Phase-2):** `gcloud storage cp` treats `[...]` as a glob, so source paths under
191     > `[Done] Video N/` (Drive) or `[Khelsutra] ?` (F:) fail with "matched no objects." The owner-box
192     > script (`scratch/copy_videos_to_gcs.ps1`) stages such paths through a bracket-free temp file; the
193     > `rclone` cloud path has no such issue.
194     ---
195
196     ## 8. Progress tracker
197
198     | Item | Status |
199     |---|---|
200     | Bucket + SA + provisioning (deploy/gcp) | ? built |
201     | Cloud worker `infer`/`train` by URI | ? built (`backend/worker`) |
202     | This data-map doc | ? 2026-06-22 |
203     | Phase 1: trajectories ? `trajectories/` (15) | ? 2026-06-22 |
204     | Phase 1: labels reconciled ? `labels/<date>_<name>` (15) | ? 2026-06-22 |
205     | Phase 1: eval manifest ? `manifests/golden.json` | ? 2026-06-22 |
206     | Phase 1: remove 7 stale `_proxy`/bare label objects | ? 2026-06-22 (MD5-verified dupes, deleted) |
207     | Phase 2: corpus videos/proxies ? `videos/<name>.mp4` | ? mapping + script done;
**1/15** demo'd (`mahadevpura_1`); 14 remain ? run `rclone` on a cloud box (preferred) or the

```

```

owner-box script |
208 | Phase 2: proxy the 8 large F: full-MP4 clips before upload | ? |
209 | Phase 3: eval/litmus default to the GCS manifest | ? |
210
211 **Definition of done:** a fresh GPU box (or CI) can run inference, the eval suite, and
Gen-0 training
212 reading **only** from `gs://khelsutra-rally-corpus` ? no `C:` / `F:` / local-Drive path
required.
213
214 ---
215
216 ## Related
217 - [`SETUP_NEW_MACHINE.md`](SETUP_NEW_MACHINE.md) ? spin up a box (storage backends in
$7).
218 - [`deploy/gcp/README.md`](../deploy/gcp/README.md) ? GCP GPU provisioning + train/infer
($5).
219 - [`VIDEO_LOCALITY_MODEL.md`](VIDEO_LOCALITY_MODEL.md) .
[`PER_VIDEO_WORKSPACE.md`](PER_VIDEO_WORKSPACE.md) ? locality tiers + `outputs/<video_id>/`.
220 - `docs/archives/past_projects/DECOUPLED_COMPUTE/` ? the cloud-engine design lineage.
221

```

6. user (2026-06-22T10:25:28.632Z)

```

1 # Setting up a new machine ? zero ? training & inference
2
3 **The canonical "spin up a box and make the models ready" runbook.** Copy-pasteable;
followable by
4 any human.
5
6 > **? COMPUTE = GCP (ADR-013, 2026-06-20). DigitalOcean is DROPPED for compute** ? use
7 > **[`deploy/gcp/README.md`](../deploy/gcp/README.md)** for the real GPU/training path
(GCP is the sole
8 > stack: DO GPU isn't enabled on the account + isn't cleanly credit-funded; Paperspace
is
9 > subscription-gated). The provider-neutral steps here (bootstrap / scratch / gpu_setup,
the suite,
10 > secret-free inference) still apply on **any** Linux box; the DigitalOcean
**provisioning** commands
11 > below are retained only as a generic example.
12
13 > **Last verified:** 2026-06-20 on a DigitalOcean Droplet (Ubuntu 24.04, 2 vCPU / 3.8
GB, **no GPU**).
14 > Bootstrap succeeded (torch 2.12.1+cpu); full mocked suite **922 passed / 11 skipped**;
a **secret-free**
15 > CPU inference (no GPU, no API key) detected **2 rallies** and persisted them to
SQLite.
16 >
17 > ? Assumes the DECOUPLED_COMPUTE PRs are merged: the CPU-mock detector seam,
`deploy/digitalocean/bootstrap.sh`,
18 > the storage layer, and the worker dispatcher (#251). The **bucket** steps ($7) are
config-driven and
19 > **verified against real DO Spaces + GCS** ? see the appendix.
20
21 ## What you'll have at the end
22
23 A machine that can: run the test suite (green), run **secret-free CPU inference**
(rallies, no GPU/keys),
24 run **real Gemini inference** (with a key), and run **Gen-0 model training** ? plus
pointers to the GPU /
25 native-detector path and bucket-driven train/infer.
26
27 ---
28
29 ## 1. Credentials to create once (least-privilege)
30

```

```

31 | # | Credential | Where | Needed for |
32 |---|---|---|---|
33 | 1 | **SSH keypair** (public key added to the provider) | `ssh-keygen -t ed25519 -f
~/ssh/rally` ? paste `.pub` into DO ? Settings ? Security | logging into the box |
34 | 2 | **DigitalOcean API token** (read+write) | DO ? API ? Tokens | `doctl` provisioning
(optional ? you can click-create) |
35 | 3 | **GitHub repo read access** | a read-only **deploy key** *or* a fine-grained
**PAT** (`Contents: Read`) | `git clone` on the box (or use the `scp` path in §3) |
36 | 4 | **`GEMINI_API_KEY`** | Google AI Studio / Vertex | **real** Gemini inference only
(§6b). The secret-free path (§6a) needs none. **Rotate before sharing.** |
37 | 5 | **Bucket creds** ? DO Spaces key/secret, or GCS SA JSON, or Drive SA JSON + folder
| provider console | bucket-driven train/infer (§7, *pending PR-B*) |
38 | 6 | **GPU box** (Paperspace free M4000 / GPU Droplet) | provider console | real
WASB/TrackNet detectors + faster runs (§8) |
39
40 **Secret hygiene:** never put a secret in the repo, a PR, or `config.json`. Set them as
env vars on the box
41 (or a git-ignored `.env`). Scope each as narrowly as the UI allows and rotate/revoke
when done.
42
43 ---
44
45 ## 2. Provision the machine
46
47 > **? COMPUTE = GCP only (2026-06-20, ADR-013 supersedes the ADR-011 split):**
48 > **BOTH training and serving run on GCP** ? provision via **`deploy/gcp/README.md`**
(GPU VM today;
49 > Cloud Run+GPU scale-to-zero for serving, ADR-012). **DigitalOcean is dropped for
compute** (DO GPU
50 > isn't enabled on the account + DO charges newer customers' cards for GPU, so it isn't
cleanly
51 > credit-funded; Paperspace is subscription-gated). The GCS bucket `gs://kheldutra-
rally-corpus` + SA +
52 > `$100` budget are the storage/cost anchors. The native WASB runner
(`detector_impl="native"`) runs the
53 > same on any Linux GPU box. **The DigitalOcean provisioning steps below are a generic
example only.**
54
55 - **CPU box** (suite + secret-free stub inference + Gen-0 training plumbing): cheapest
**Ubuntu 24.04**
56 Droplet, **2 vCPU / 4 GB**, region nearest you, **attach the SSH key** (#1). ~$0.06/hr
? destroy when done.
57 - **GPU box** (real WASB/TrackNet detectors ? real trajectory extraction + training): a
**DO GPU Droplet**.
58 ? You **cannot add a GPU to a CPU Droplet** ? it's a separate machine type, in **GPU
regions**
59 (`nyc2`/`tor1`/`atl1`, **not** `blr1`), with **dual local NVMe**. **RTX 4000 Ada (20
GB, ~$0.76/hr)** is
60 plenty for WASB + Gen-0; the `gpu-*-base` image preinstalls the NVIDIA driver. **Co-
locate the Space in the
61 GPU box's region** for cheap corpus-pull.
62
63 CLI provision (after `doctl auth init` with token #2 ? **verify size/image slugs first**
via `doctl compute size list`):
64 ```bash
65 # CPU box:
66 doctl compute droplet create rally-box --image ubuntu-24-04-x64 --size s-2vcpu-4gb \
67 --region blr1 --ssh-keys <your-key-fp> --wait
68 # GPU box (RTX 4000 Ada):
69 doctl compute droplet create rally-gpu --image gpu-h100x1-base --size gpu-4000adax1-20gb
\
70 --region tor1 --ssh-keys <your-key-fp> --wait
71 doctl compute droplet list # note the IP -> ssh -i ~/.ssh/rally root@<IP>
72 ```
73

```



```

74    ---
75
76    ## 3. Get the code onto the box
77
78    **Option A ? git clone** (private repo, so authenticate with #3):
79    ```bash
80    ssh -i ~/.ssh/rally root@<IP>
81    git clone git@github.com:avidullu/badminton-highlight-indexer.git ~/rally # via deploy
key
82    # or: git clone https://<PAT>@github.com/avidullu/badminton-highlight-indexer.git
~/rally
83    cd ~/rally
84    ```
85
86    **Option B ? push a local checkout** (no GitHub access needed; what the maintainer
used):
87    ```bash
88    # on your machine, from the repo root:
89    git archive --format=tar.gz -o /tmp/rally.tgz HEAD
90    scp -i ~/.ssh/rally /tmp/rally.tgz root@<IP>:/root/
91    ssh -i ~/.ssh/rally root@<IP> 'mkdir -p ~/rally && tar -xzf ~/rally.tgz -C ~/rally'
92    ```
93
94    ---
95
96    ## 4. Bootstrap (one command)
97
98    ```bash
99    cd ~/rally
100    ./deploy/digitalocean/bootstrap.sh # CPU box
101    # or
102    ./deploy/digitalocean/bootstrap.sh --gpu # GPU box (installs CUDA torch)
103    ```
104
105    It mirrors CI: installs `ffmpeg` + opencv runtime, creates `.venv`, installs CPU-or-CUDA
`torch`, then
106    `pip install -e .[dev]`, and prints a device check (`cuda_available True/False`).
107
108    <details><summary>Manual equivalent (if you can't run the script)</summary>
109
110    ```bash
111    sudo apt-get update
112    sudo apt-get install -y python3 python3-venv python3-pip git ffmpeg libgl1
113    sudo apt-get install -y libglib2.0-0t64 || sudo apt-get install -y libglib2.0-0
114    python3 -m venv .venv && . .venv/bin/activate && python -m pip install -U pip
115    pip install torch torchvision --index-url https://download.pytorch.org/whl/cpu # or
.../whl/cu124 on GPU
116    pip install -e .[dev]
117    ```
118    </details>
119
120    ---
121
122    ## 5. Verify the machine (the smoke gate)
123
124    ```bash
125    . .venv/bin/activate
126    python -m pytest -q # expect all green (922 passed / 11 skipped on the current
tree)
127    ```
128
129    A green suite proves the whole pipeline runs on this box with **no GPU, no WSL, no
bucket, no API key**.
130
131    ---

```

```

132     ## 6. Run inference
133
134     ### 6a. Secret-free CPU inference (no GPU, no API key) ? the "mock a GPU with a CPU"
path
135
136     Drop a permissive smoke `config.json` in the repo root (the CLI reads `./config.json`
from the cwd):
137     ```bash
138     cd ~/rally && cat > config.json <<'JSON'
139     {
140         "sport": "badminton",
141         "indexing": { "default_segmenter": "wasb_hybrid", "detector_impl": "stub",
"skip_ai_handoff": true },
142         "validation": { "min_fps": 0, "min_blur_threshold": 0, "max_scene_cuts_per_minute":
1000,
143                         "relevance": { "court_pixels_min_ratio": 0.0 } }
144     }
145     JSON
146
147     # a throwaway 12s 1280x720 test clip
148     ffmpeg -y -f lavfi -i testsrc=duration=12:size=1280x720:rate=30 -pix_fmt yuv420p
/root/sample.mp4
149
150     # run ? env -u drops any Gemini key to prove none is needed
151     env -u GEMINI_API_KEY RALLY_DETECTOR_IMPL=stub \
152         indexer --video-path /root/sample.mp4 --segmenter wasb_hybrid --output-dir ~/rally/out
153     ```
154     Expect: `detector_impl=stub` ? CPU mock runner`, `Phase 3 SKIPPED (fully-local)`, and
155     `Indexed 2 play segments` in `~/rally/out/sports_indexer.db`. *(`detector_impl=stub`
swaps the GPU+WSL
156     detector for a deterministic CPU one; `skip_ai_handoff=true` makes candidate windows the
rallies, so no
157     Gemini handoff happens.)*
158
159     ### 6b. Real inference (Gemini segmenter)
160
161     ```bash
162     export GEMINI_API_KEY=""           # secret stays in the env, never in
the repo
163     # use a real badminton clip (must pass validation: ?1280x720, ?30fps, a green court,
etc.)
164     indexer --video-path /root/match.mp4 --segmenter gemini --output-dir ~/rally/out
165     #   or the hybrid: --segmenter wasb_hybrid (WASB needs a GPU box, $8) with
skip_ai_handoff=false
166     ```
167
168     ---
169
170     ## 7. Object storage ? bucket-driven train/infer (S3-compatible / GCS / Drive)
171
172     The worker (`python -m backend.worker`) reads/writes by URI, so you choose a backend
with config +
173     credentials ? no code change. A bare path / `local://` is a passthrough (today's
behaviour); `s3://` and
174     `gcs://` go to a bucket. Install only the extra you use (cloud SDKs are lazy-imported).
175
176     > Verified end-to-end 2026-06-20: the worker bucket round-trip (pull video ? infer ?
push outputs) passed
177     > against real DigitalOcean Spaces (sgp1), a real GCS client (emulator), and
MinIO.
178
179     ### 7a. S3-compatible ? AWS S3 / DigitalOcean Spaces / Cloudflare R2 / MinIO
180
181     One backend for all four ? they differ only by `endpoint_url` + region. `pip install -e
.[storage-s3]`.

```

```

182
183  **Credentials** ? `~/aws/credentials` (boto3's default chain; **never** in the repo):
184  ```ini
185  [default]
186  aws_access_key_id = <key>
187  aws_secret_access_key = <secret>
188  ```
189  **Config** ? `config.json` `storage.s3`:
190  ```jsonc
191  "storage": { "backend": "local", "s3": {
192    "endpoint_url": "https://sgp1.digitaloceanspaces.com", // DO Spaces (VERIFIED);
region = the Space's, e.g. sgp1
193    "region": "sgp1"
194    // AWS S3 : omit endpoint_url; set region (e.g. ap-south-1)
195    // R2      : endpoint_url https://<account>.r2.cloudflarestorage.com ; region "auto"
196    // MinIO   : endpoint_url http://host:9000 ; region us-east-1 ; "addressing_style":
"path"
197  }}
198  ```
199  **Run** (secret-free flavor; use `--segmenter gemini` for real detection):
200  ```bash
201  env -u GEMINI_API_KEY RALLY_DETECTOR_IMPL=stub python -m backend.worker infer \
202    --video-uri s3://<bucket>/videos/clip.mp4 --out-uri s3://<bucket> \
203    --segmenter wasb_hybrid --set indexing.skip_ai_handoff=true
204  ```
205
206  ### 7b. Google Cloud Storage
207
208  `pip install -e [storage-gcs]`. **Credentials** ? a service-account JSON (SA with
*Storage Object Admin*):
209  ```bash
210  export GOOGLE_APPLICATION_CREDENTIALS=/path/to/sa.json
211  ```
212  **Config** ? `storage.gcs.project` (no endpoint ? default `googleapis.com`):
213  ```jsonc
214  "storage": { "backend": "local", "gcs": { "project": "<gcp-project-id>" } }
215  ```
216  Same `worker infer` command with `gcs://<bucket>/?` URIs. *(On-box test with no GCP
creds:
217  `pip install gcp-storage-emulator && gcp-storage-emulator start --port 9023 --in-memory`
+
218  `export STORAGE_EMULATOR_HOST=http://127.0.0.1:9023`.)*)
219
220  ### 7c. Google Drive *(corpus source ? assumes a MOUNTED Drive)*
221
222  **Baked-in assumption: you mount Google Drive for Desktop.** That's a fair requirement
for using
223  Drive files locally ? it needs no service account, no API quota, no OAuth, and Drive
streams + caches
224  bytes on access. Once mounted, reference files by a path under your *My Drive* with the
`gdrive://`
225  scheme ? a portable, OS-agnostic alias over the mount (so the same URI works on any
mounted box):
226  ```bash
227  python -m backend.worker infer \
228    --video-uri "gdrive://Sharing/Annotation Videos/Golden Label Videos Request - June
15/[Done] Video 4 - Badminton/GX020094_proxy.mp4" ?
229  ```
230  The backend resolves the mount root automatically ? Windows `G:\My Drive`, macOS
231  `~/Library/CloudStorage/GoogleDrive-<acct>/My Drive` ? or set it explicitly (it's a
path, not a secret):
232  ```bash
233  export GDRIVE_MOUNT_ROOT="G:\My Drive"          # or: storage.gdrive.mount_root in
config
234  ```

```

```

235     The shared golden-corpus folder `drive/folders/1sHGL7iacnmM80ChT-p2cKy638gK504T_` mounts
at
236     `G:\My Drive\Sharing\Annotation Videos\Golden Label Videos Request - June 15\`. *(You
can also just pass
237     the raw `G:\My Drive\` path ? the default `local` backend handles it identically;
`gdrive://` only adds
238     mount-root portability.)* If Drive isn't mounted the backend fails fast with a clear
"mount your Drive"
239     error. ? **A headless GPU VM has no mount** ? there, stage Drive?bucket and read
`gcs://`, or `gdown
240     <file-id> -O clip.mp4` to a local path.
241
242     **Bucket layout (for the bucket backends):** `videos/ labels/ manifest.json weights/
outputs/<video_id>/`
243     (see `docs/DECOUPLED_COMPUTE/04`).
244
245     ---
246
247     ## 8. Run training (Gen-0 rally head)
248
249     CPU-runnable, minutes ? but it needs a **labeled corpus** (videos + `.rallies.csv`
labels + a manifest),
250     which a fresh box doesn't have. Bring it via $7's bucket (`worker train`, pending) or
`scp` it up, then:
251     ```bash
252     python -m training.gen0.harness --manifest <manifest.json> --floor <baseline> --version
0.1.0
253     ```
254     The harness runs train ? eval ? ship-gate and emits a versioned weight bundle. *(The
Gen-2 VLM is a
255     separate, GPU-heavy, separately-budgeted effort ? out of scope here.)*
256
257     ---
258
259     ## 9. GPU box ? port procedure (NVMe + native WASB)
260
261     The GPU box is **cattle, not a pet**: durable state lives in the bucket + git, so a
fresh box is running in
262     minutes. Port = provision ($2) ? code on ($3) ? NVMe ? `bootstrap.sh --gpu` ?
`gpu_setup.sh` ? pull corpus.
263
264     ```bash
265     # on the box, after $3 put the code at ~/rally:
266     cd ~/rally
267     sudo bash deploy/digitalocean/mount_scratch.sh && export TMPDIR=/scratch #
detect+mount the spare NVMe
268     ./deploy/digitalocean/bootstrap.sh --gpu # CUDA
torch; expects cuda_available True
269     . .venv/bin/activate && pip install -e .[storage-s3]
270     bash deploy/digitalocean/gpu_setup.sh # native
WASB conda env (no WSL) ? M3.5 enabler
271     # secrets (the only two non-reproducible items): Gemini key + bucket creds
272     mkdir -p /root/.keys && printf '%s' '<rotated-key>' > /root/.keys/GEMINI_API_KEY &&
chmod 600 /root/.keys/GEMINI_API_KEY
273     install -m600 /dev/stdin ~/.aws/credentials <<'INI'
274     [default]
275     aws_access_key_id = <key>
276     aws_secret_access_key = <secret>
277     INI
278     ```
279
280     - **NVMe scratch** (`mount_scratch.sh`): DO GPU Droplets ship a second raw NVMe; the
script formats+mounts it
281     at `/scratch` and the pipeline's sync-first decode runs there (`export
TMPDIR=/scratch` or `--scratch /scratch`).

```

```

282     Ephemeral, not in fstab. On a CPU box it just makes `/scratch` on the boot disk.
283     - **Native WASB** (`gpu_setup.sh`): replicates `docs/TRACKNET_WSL_SETUP.md`'s conda env
**natively on Linux**
284     (no WSL) ? the M3.5 enabler. ? Needs `wasb_badminton_best.pth.tar` present and the
**WASB-C3** weight
285     provenance cleared before sharing any derived model.
286     - **The detector itself:** real `--segmenter wasb_hybrid` extraction needs the Linux-
native `DetectorRunner`
287     (tracked M3.5, `docs/DECOUPLED_COMPUTE/04`). Until it lands, validate the train/infer
plumbing with
288     `detector_impl=stub`.
289     - **Teardown** = `doctl compute droplet delete rally-gpu`. Re-spin tomorrow identically.
290
291     ---
292
293     ## 10. Teardown
294
295     ```bash
296     rm -f ~/rally/config.json ~/sample.mp4          # remove the throwaway smoke artifacts
297     doctl compute droplet delete rally-box          # or destroy in the console
298     ```
299     Revoke any temporary token/key (#2?#5) and delete the SSH key from the provider when the
box is gone.
300
301     ---
302
303     ## Appendix ? verification log
304
305     | Step | Command | Result (2026-06-20, DO Ubuntu 24.04, 2 vCPU, no GPU) |
306     |---|---|---|
307     | Bootstrap | `bootstrap.sh` | ? torch 2.12.1+cpu, deps + editable build |
308     | Smoke | `pytest -q` | ? 922 passed, 11 skipped (49s) |
309     | Secret-free infer (local) | $6a | ? 2 rallies, persisted to SQLite, `GEMINI_API_KEY`
unset |
310     | **S3 worker round-trip ? real DO Spaces (sgp1)** | $7a | ? pulled
`s3://rally-e2e/videos/?` ? 2 rallies ? pushed `outputs/<md5>/intervals.csv` |
311     | **GCS worker round-trip** (emulator) | $7b | ? pulled `gcs:///?` ? 2 rallies ? pushed
`gcs:///?/outputs/<md5>/?` |
312     | S3 backend vs MinIO (put/get/list/stat + presigned GET) | $7a | ? |
313

```

7. assistant (2026-06-22T10:25:33.430Z)

Now let me read the deploy/gcp/README.md and the worker code.

8. user (2026-06-22T10:25:34.380Z)

```

1     # GCP provisioning runbook ? DECOUPLED_COMPUTE GPU + storage (asia-south1)
2
3     > **? GCP is the SOLE compute stack (ADR-013, 2026-06-20).** DigitalOcean is **dropped
for compute**
4     > (DO GPU Droplets aren't enabled on the account + aren't cleanly credit-funded ? DO
charges newer
5     > customers' cards for GPU; Paperspace is subscription-gated). **All training + serving
runs on GCP.**
6     > The `deploy/digitalocean/` scripts are kept only as **provider-agnostic** Linux-GPU
setup that this
7     > runbook reuses. ? Capacity note: asia-south1 (Mumbai) GPUs are frequently stocked-out
? sweep zones /
8     > fall back to asia-southeast1 (Singapore) etc.; weights still land in the asia-south1
bucket.
9
10    Co-located **GPU VM + GCS bucket in ONE region** (asia-south1 / Mumbai) for the first
real
11    (non-mock) WASB training + inference. This is the **DO?GCP pivot** (ADR-010):

```

DigitalOcean GPU

```
12     Droplets are North-America-only (no Singapore/India GPU), so we moved GPU + storage to
GCP, which
13     has GPUs in asia-south1 (Mumbai) and asia-southeast1 (Singapore). Owner picked
**Mumbai** (closest
14     to Bangalore ? lowest latency).
15
16     > **Cost discipline (read first):** a running GPU VM is the only meaningful cost. Use
the cheapest
17     > adequate GPU (**T4**), **STOP or DELETE the VM the moment a run finishes** (a stopped
VM bills only
18     > for its disk), and keep the **$100 budget alert** live. `teardown.sh` switches
everything off.
19
20     ---
21
22     ## 0. Live resources (created 2026-06-20 ? the "switch-off" inventory)
23
24     | Resource | Identifier | Cost when idle | How to switch off |
25     |---|---|---|---|
26     | Project | `gen-lang-client-0306026826` (billing **Khelsutra Billing**
`01420D-419EE0-53D83C`) | ? | n/a (shared Gemini project) |
27     | GCS bucket | `gs://khelsutra-rally-corpus` (asia-south1, UBLA) | ~$0.02/GB/mo storage
| `gcloud storage rm -r` (? data loss) |
28     | Service account | `rally-worker@gen-lang-client-0306026826.iam.gserviceaccount.com`
(`roles/storage.objectAdmin`) | $0 | `gcloud iam service-accounts delete` |
29     | SA key | `~/.gcp/rally-worker-sa.json` (local only, **never** committed) | $0 | delete
the file + the key |
30     | Budget alert | `rally-100usd-guard` ? $100, alerts at 50/90/100% | $0 | `gcloud
billing budgets delete` |
31     | **GPU VM** | **NOT created** ? blocked on `GPUS_ALL_REGIONS` quota (see §2) |
**~$0.35/hr (T4) running, ~$0.01/hr stopped** | `teardown.sh` (stop or delete) |
32     | APIs enabled | serviceusage, compute, cloudbilling, billingbudgets, iam,
cloudresourcemanager | $0 | leave on (free) |
33
34     Run `bash deploy/gcp/teardown.sh` (see §6) to stop/delete the billable bits between
sessions.
35
36     ---
37
38     ## 1. One-time project bootstrap (owner, console ? already done 2026-06-20)
39
40     A bare Gemini/AI-Studio project has the **Service Usage API disabled**, which blocks
*all* gcloud
41     provisioning (gcloud's own `services enable` needs that API ? chicken-and-egg). These
were enabled
42     in the console; recorded here so a fresh project can be bootstrapped the same way:
43
44     1. **Service Usage API** ?
https://console.developers.google.com/apis/api/serviceusage.googleapis.com/overview?project=gen-
lang-client-0306026826
45     2. **Link a billing account** (a payment method) ?
https://console.cloud.google.com/billing/linkedaccount?project=gen-lang-client-0306026826
46     3. After (1), the rest enable via gcloud (done): `gcloud services enable
compute.googleapis.com cloudbilling.googleapis.com billingbudgets.googleapis.com
iam.googleapis.com cloudresourcemanager.googleapis.com`
47
48     `provision.sh` (§5) is idempotent and re-runs all of step (3) + the bucket/SA/budget.
49
50     ---
51
52     ## 2. ? GPU quota ? the one remaining owner action
53
54     New GCP projects start at **`GPUS_ALL_REGIONS` = 0**, the *global* cap that gates GPU
creation even
```

```

55     when a regional per-type quota is non-zero. Confirmed 2026-06-20:
56
57     ```
58     asia-south1  NVIDIA_T4_GPUS = 1    NVIDIA_L4_GPUS = 1    (regional: OK)
59     GLOBAL      GPUS_ALL_REGIONS = 0    ? BLOCKS the VM
60     ```
61
62     **Owner:** IAM & Admin ? **Quotas** ? filter ***GPUs (all regions)*** ? Edit ? request
63     **1** ? submit.
64     (For a single T4 this is usually approved in minutes; sometimes a short review.) Re-
65     check:
66
67     ```bash
68     gcloud compute project-info describe --project gen-lang-client-0306026826 \
69     --flatten="quotas[]" --format="table(quotas.metric,quotas.limit)" | grep
70     GPUS_ALL_REGIONS
71     ```
72
73     When that reads `1`, run $3.
74
75     > **? Spot does NOT skip the quota.** TESTED 2026-06-20: a **Spot** T4 create also fails
76     with
77     > `Quota 'GPUS_ALL_REGIONS' exceeded. Limit: 0.0 globally` ? `GPUS_ALL_REGIONS` gates
78     **both**
79     > on-demand AND preemptible GPUs, so the increase above is required either way. (A
80     pending request
81     > was filed via the Cloud Quotas API: `POST ?/quotaPreferences` `GPUS-ALL-REGIONS-per-
82     project`
83     > ? 1.) **Note (2026-06-20, ADR-013 supersedes the ADR-011 split): GCP is the SOLE
84     compute stack ?
85     > BOTH training and serving run on GCP; DigitalOcean is dropped for compute.** Spot is
86     still cheaper
87     > once the quota is granted, and the resumable cache makes preemption recoverable.
88
89     ---
90
91     ## 3. Create the GPU VM (after quota ? 1)
92
93     > **? Recommended: `bash deploy/gcp/create_gpu_vm.sh`** ? the one-command way. It sweeps
94     zones for
95     > L4 capacity (Mumbai ? Singapore), uses the driver-ready image, attaches the `rally-
96     worker` SA, and
97     > **always bakes in the Ops Agent** (`--metadata-from-file startup-
98     script=deploy/gcp/ops-agent-startup.sh`)
99     > so Observability/GPU graphs populate automatically (no more "Ops Agent missing"). It
100     prints the
101     > zone + the SSH/DELETE commands. `RALLY_GPU=t4` for a T4; `RALLY_VM_NAME=?` to rename.
102     The manual
103     > command below is kept for reference / customization.
104
105     T4 is plenty for WASB + Gen-0. A local SSD gives a fast `/scratch`. *(The native runner
106     defaults to
107     **streaming** ? it decodes frames in-process, skipping the slow cv2 PNG extraction; bit-
108     identical to disk,
109     verified on a real GPU 2026-06-20. Set `indexing.wasb.stream_video=false` only if a
110     container/codecs cv2
111     can't seek; `WASB_SCRATCH` still backs the disk path.)*
112
113     ```bash
114     gcloud compute instances create rally-gpu \
115     --project=gen-lang-client-0306026826 \
116     --zone=asia-south1-a \
117     --machine-type=n1-standard-8 \
118     --accelerator=type=nvidia-tesla-t4,count=1 \
119     --maintenance-policy=TERMINATE --provisioning-model=STANDARD \

```

```

103     --image-family=ubuntu-2404-lts-amd64 --image-project=ubuntu-os-cloud \
104     --boot-disk-size=120GB --boot-disk-type=pd-balanced \
105     --local-ssd=interface=NVME \
106     --service-account=rally-worker@gen-lang-client-0306026826.iam.gserviceaccount.com \
107     --scopes=cloud-platform
108     ```
109
110     **Spot variant (cheaper; still needs the $2 quota ? Spot is NOT a quota bypass):** same
command with
111     `--provisioning-model=SPOT` (replacing `STANDARD`). Add `--instance-termination-
action=STOP` to keep
112     the disk on preemption so a re-run resumes. Cheaper, can be preempted.
113
114     > **Validated image (2026-06-20):** `--image-family=common-cu129-ubuntu-2204-nvidia-580
115     > --image-project=deeplearning-platform-release` ships the NVIDIA driver (no install-on-
boot wait;
116     > `nvidia-smi` works immediately) ? used for the first real L4 run. `gpu_setup.sh`
installs miniconda
117     > on it. **T4 was capacity-stocked-out in `asia-south1-a`; an L4 (`g2-standard-4`, no
`--accelerator`
118     > flag ? L4 is built into g2) in `asia-south1-c` landed** ? try L4/other zones if T4 is
unavailable.
119
120     ### 3b. Observability ? Ops Agent (GPU graphs + logs in the console)
121     The VM **Observability ? GPU** graphs + Logs need the **Ops Agent** (the console's OS-
policy install is
122     blocked on DLVM images). Install it directly + grant the worker SA the write roles (it
otherwise only has
123     storage), then GPU/NVML metrics + syslog flow to Cloud Monitoring/Logging (tab populates
in ~2-3 min):
124     ```bash
125     P=gen-lang-client-0306026826; SA=rally-worker@$P.iam.gserviceaccount.com
126     # 1. ENABLE the APIs (bare Gemini project has them OFF ? the agent installs but its
exports get
127     #   PermissionDenied/SERVICE_DISABLED until these are on; THIS was the empty-graphs
cause 2026-06-20):
128     gcloud services enable logging.googleapis.com monitoring.googleapis.com --project $P
129     # 2. let rally-worker publish metrics + logs:
130     gcloud projects add-iam-policy-binding $P --member="serviceAccount:$SA"
--role=roles/monitoring.metricWriter --condition=None
131     gcloud projects add-iam-policy-binding $P --member="serviceAccount:$SA"
--role=roles/logging.logWriter --condition=None
132     # 3. on the box (created with --scopes=cloud-platform):
133     curl -sSO https://dl.google.com/cloudagents/add-google-cloud-ops-agent-repo.sh && sudo
bash add-google-cloud-ops-agent-repo.sh --also-install
134     # (if the agent was installed BEFORE the APIs were enabled: `sudo systemctl restart
google-cloud-ops-agent`)
135     ```
136     > Best: bake `curl ?add-google-cloud-ops-agent-repo.sh|bash --also-install` into the VM
137     > `--metadata=startup-script` so observability is automatic on every box.
138
139     > **Long remote steps must be detached.** A plain `ssh ? "<long cmd>"` dies if the SSH
drops (seen
140     > mid-build, 2026-06-20) ? run them `nohup`'d on the box (`nohup bash setup.sh >log 2>&1
&`) and poll
141     > the log, so a dropped connection can't kill the install.
142
143     > If `asia-south1-a` reports no T4 capacity, try `-b` / `-c`. The Ubuntu image needs the
NVIDIA
144     > driver: either add `--metadata=install-nvidia-driver=True` style startup, or after SSH
run GCP's
145     > driver installer, **or** use a Deep-Learning-VM image (`--image-family=common-
cu124-ubuntu-2204`
146     > `--image-project=deeplearning-platform-release`) which ships the driver. Confirm
`nvidia-smi` works

```



```

147     > before §4.
148
149     SSH: `gcloud compute ssh rally-gpu --zone=asia-south1-a`.
150
151     ---
152
153     ## 4. Port the repo + WASB env onto the box
154
155     The `deploy/digitalocean/` scripts are **provider-agnostic** (they detect the local NVMe
/ install
156     torch); reuse them on GCP:
157
158     ```bash
159     # from your machine ? ship the repo, BAKING the source commit so the trained bundle is
160     # provenance-stamped. A git-archive tarball carries no .git, so training/gen0/harness.py
would
161     # otherwise record git_sha:"unknown" (seen on the 2026-06-20 L4 run). It reads, in
order:
162     # live `git rev-parse` ? $RALLY_GIT_SHA ? a GIT_SHA file at the repo root ? "unknown".
163     git rev-parse HEAD > /tmp/GIT_SHA
164     git archive --format=tar.gz --add-file=/tmp/GIT_SHA -o /tmp/rally.tgz HEAD # GIT_SHA ?
archive root (git ?2.38)
165     gcloud compute scp /tmp/rally.tgz rally-gpu:/root/ --zone=asia-south1-a
166     gcloud compute ssh rally-gpu --zone=asia-south1-a -- 'mkdir -p ~/rally && tar -xzf
~/rally.tgz -C ~/rally'
167     # (Alternative, if not baking the file: `export RALLY_GIT_SHA=$(git rev-parse HEAD)` on
the box
168     # in the same shell that runs `python -m backend.worker train ?`.)
169
170     # on the box:
171     cd ~/rally
172     sudo bash deploy/digitalocean/mount_scratch.sh && export TMPDIR=/scratch # detects the
local NVMe
173     ./deploy/digitalocean/bootstrap.sh --gpu # venv +
torch cu124 (confirm cuda True)
174     . .venv/bin/activate && pip install -e .[storage-gcs] # GCS backend
175     bash deploy/digitalocean/gpu_setup.sh # native
`wasb` conda env + WASB-SBDT
176
177     # secrets (NOT in repo): the GCS SA JSON + a rotated Gemini key
178     gcloud compute scp ~/.gcp/rally-worker-sa.json rally-gpu:/root/.gcp/sa.json --zone=asia-
south1-a
179     # on the box: export GOOGLE_APPLICATION_CREDENTIALS=/root/.gcp/sa.json
180     mkdir -p /root/.keys && printf '%s' '<ROTATED_GEMINI_KEY>' > /root/.keys/GEMINI_API_KEY
&& chmod 600 /root/.keys/GEMINI_API_KEY
181     ```
182
183     WASB weights (`wasb_badminton_best.pth.tar`) must be present at
184     `~/models/WASB-SBDT/pretrained_weights/` (gpu_setup.sh does NOT fetch them ? stage from
Drive/bucket).
185     Point the native runner at them: `export WASB_WEIGHTS=~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar`
186     and `export WASB_PYTHON=$(conda run -n wasb which python)` (or the env's python path).
See
187     `docs/TRACKNET_WSL_SETUP.md` for the WASB env. **WASB-C3:** weight provenance is an open
legal item
188     before sharing any trajectories/model externally ? internal extraction + Gen-0 is fine.
189
190     ---
191
192     ## 5. Stage the corpus + run the first REAL train/infer
193
194     Config for GCS (no secrets in config ? auth via `GOOGLE_APPLICATION_CREDENTIALS`):
195
196     ```json

```

```

197     { "sport": "badminton",
198       "indexing": { "default_segmenter": "wasb_hybrid", "detector_impl": "native" },
199       "storage": { "backend": "local", "gcs": { "project": "gen-lang-client-0306026826" } }
200     }
201
202     ```bash
203     # stage the 7 Done videos + 7 labels into the bucket (from the co-located box ? fast):
204     #   videos -> gs://khelsutra-rally-corpus/videos/   labels -> gs://khelsutra-rally-
corpus/labels/
205     # (owner handles the actual copy; see "Upload options" below)
206
207     # FIRST REAL training (real WASB trajectories, native runner):
208     RALLY_DETECTOR_IMPL=native python -m backend.worker train \
209       --corpus-uri gcs://khelsutra-rally-corpus --out-uri gcs://khelsutra-rally-corpus \
210       --version 0.1.0 --trajectory-source detector --segmenter wasb_hybrid --config
config.json
211
212     # FIRST REAL inference on the held-out Video 1 (GX010135_proxy.mp4, no labels):
213     python -m backend.worker infer \
214       --video-uri gcs://khelsutra-rally-corpus/videos/GX010135_proxy.mp4 \
215       --out-uri gcs://khelsutra-rally-corpus --segmenter wasb_hybrid --config config.json
216     ```
217
218     ### Upload options (how to get a video in + run inference)
219     The worker resolves `--video-uri` via the storage layer ? pick whichever fits:
220     - **GCS bucket (recommended, co-located):** `gcs://khelsutra-rally-
corpus/videos/clip.mp4`. Upload with
221       `gcloud storage cp clip.mp4 gs://khelsutra-rally-corpus/videos/`.
222     - **S3 / D0 Spaces / R2:** `s3://bucket/clip.mp4` (install `[storage-s3]`, creds in
`~/aws/credentials`).
223     - **Local upload:** scp the file to the box and pass a bare path /
`local:///root/clip.mp4` (identity passthrough, no copy).
224     - **Google Drive (assumes Drive is MOUNTED):** mount Google Drive for Desktop ? a fair
requirement to
225       use Drive files locally ? and reference them with `gdrive://<path-under-My-Drive>`.
The backend resolves
226       it against your local mount (auto-detected: Windows `G:\My Drive`, macOS
227       `~/Library/CloudStorage/GoogleDrive-<acct>/My Drive`; override with
`GDRIVE_MOUNT_ROOT` or
228       `storage.gdrive.mount_root`), then it's plain local I/O ? **no service account, no
API, no extra install.**
229     E.g. the golden-corpus folder `~/drive/folders/1sHGL7iacnmM80ChT-p2cKy638gK504T_`
mounts at
230       `G:\My Drive\Sharing\Annotation Videos\Golden Label Videos Request - June 15`, so
231       `gdrive://Sharing/Annotation Videos/Golden Label Videos Request - June 15/[Done] Video
4 - Badminton/GX020094_proxy.mp4`
232       (or just the raw `G:\My Drive\?` path ? the `local` backend handles that too). ? A
**headless GPU VM has
233       no mount** ? there, stage Drive?bucket and read `gcs://`, or `gdown <file-id> -O
clip.mp4` to a local path.
234
235     ---
236
237     ## 6. Switch off between sessions (cost control)
238
239     > **? DELETE, don't just STOP.** A *stopped* VM halts compute billing but **still bills
for its
240     > persistent disk** (pd-ssd ? $0.17/GB/mo ? a 200 GB boot disk ? **$34/mo**). Only
**delete** reaches
241     > $0. So `teardown.sh` **defaults to `delete`** (owner cost rule, 2026-06-20). The boot
disk has
242     > `autoDelete=true`, so delete reclaims it too ? verified 2026-06-20.
243
244     ```bash

```

```

245     bash deploy/gcp/teardown.sh          # DELETE the VM (default; $0 ? removes VM + boot
disk)
246     bash deploy/gcp/teardown.sh delete    # same (explicit)
247     bash deploy/gcp/teardown.sh stop      # STOP only ? keeps the disk ? STILL BILLS
(~$/mo!); fast resume
248     bash deploy/gcp/teardown.sh all        # delete VM + SA + budget (keeps the bucket/data)
249     # cross-region VM: set the zone (teardown defaults to asia-south1-a)
250     RALLY_GCP_ZONE=asia-southeast1-a bash deploy/gcp/teardown.sh delete
251     ```
252
253     **After deleting, audit for orphans** (a stray disk bills even with no VM):
254     `gcloud compute instances list` (empty) and `gcloud compute disks list` (no unattached
disk).
255
256     The **bucket is never auto-deleted** (it holds the corpus). Delete it explicitly only
when done:
257     `gcloud storage rm -r gs://khehsutra-rally-corpus`.
258

```

9. user (2026-06-22T10:25:35.132Z)

```

1     """Worker dispatcher ? pull from storage ? run the pipeline ? push outputs back.
2
3     python -m backend.worker infer --video-uri <uri> --out-uri <prefix> \
4         [--segmenter wasb_hybrid] [--config config.json] [--set k=v ...] [--scratch DIR]
5     python -m backend.worker train ...          # Gen-0 train-from-bucket (follow-up)
6
7     The URI scheme selects the storage backend via ``backend.storage`` (a bare path /
``local:``/``
8     is an identity passthrough; ``s3:``/`` / ``gcs:``/`` is an object store), so existing
local runs
9     are byte-for-byte unchanged. Combine with ``RALLY_DETECTOR_IMPL=stub`` and
10    ``--set indexing.skip_ai_handoff=true`` for a **secret-free, GPU-free** run (no Gemini
key) ?
11    the path validated end-to-end on DigitalOcean (``docs/SETUP_NEW_MACHINE.md``).
12
13    This is a thin orchestrator: it composes the SAME building blocks the ``backend.main``
CLI uses
14    (validator registry ? segmenter ? report), never a forked pipeline.
15    """
16    from __future__ import annotations
17
18    import argparse
19    import csv
20    import json
21    import logging
22    import os
23    import sys
24    import tempfile
25    from typing import Any, List
26
27    from backend import storage
28    from backend.compute import ComputeJobSpec, get_compute_backend
29    from backend.config import (
30        StartupCheckError,
31        enforce_startup_checks,
32        load_config,
33        probe_cuda_available,
34    )
35    from backend.infrastructure.database import Database
36    from backend.pipeline.segmenters.base import segmenter_registry
37    from backend.pipeline.validators.base import registry as validator_registry
38    from backend.reporting import emit_indexer_report
39    from backend.results import Failure
40    from backend.utils.ffmpeg import ffmpeg_bin # P2a: single ffmpeg/ffprobe resolver

```

```

41     from backend.utils.hashing import compute_video_id
42
43     logger = logging.getLogger("backend.worker")
44
45
46     def _coerce(v: str) -> Any:
47         """Coerce a ``--set`` string value to bool/int/float where it round-trips, else
48         str."""
49         low = v.lower()
50         if low in ("true", "false"):
51             return low == "true"
52         for cast in (int, float):
53             try:
54                 return cast(v)
55             except ValueError:
56                 pass
57         return v
58
59     def _apply_overrides(config: Any, sets: List[str]) -> None:
60         """Apply ``a.b.c=value`` overrides onto the typed config (e.g.
61         indexing.skip_ai_handoff=true)."""
62         for kv in sets or []:
63             key, sep, val = kv.partition("=")
64             if not sep:
65                 raise ValueError(f"--set expects k=v, got {kv!r}")
66             parts = key.split(".")
67             obj = config
68             for p in parts[:-1]:
69                 obj = getattr(obj, p)
70             setattr(obj, parts[-1], _coerce(val))
71
72     def _write_intervals_csv(segments: List[dict], path: str) -> None:
73         """Decimal-second ``[start,end)`` rallies + shot count / ending reason ? the join-
74         friendly
75         artifact (same schema as the rally-annotator labels)."""
76         with open(path, "w", newline="") as f:
77             w = csv.writer(f)
78             w.writerow(["start", "end", "shot_count", "ending_reason"])
79             for s in segments:
80                 w.writerow([
81                     f'{float(s.get("start_time", 0.0)):.3f}',
82                     f'{float(s.get("end_time", 0.0)):.3f}',
83                     s.get("shot_count", ""),
84                     s.get("ending_reason", s.get("shot_count_confidence", "")),
85                 ])
86
87     def _write_report_json(video_id: str, segments: List[dict], status: str, path: str) ->
88     None:
89         with open(path, "w") as f:
90             json.dump({
91                 "video_id": video_id,
92                 "status": status,
93                 "segment_count": len(segments),
94                 "segments": [{"start": float(s.get("start_time", 0.0)),
95                             "end": float(s.get("end_time", 0.0))} for s in segments],
96             }, f, indent=2)
97
98     def _run_startup_checks(config: Any) -> bool:
99         """M5 per-profile startup checks for the worker (the compute process, so it probes
100         CUDA
101         best-effort). Logs warnings; returns False on a FATAL coherence error so the caller

```

```

aborts
101         cleanly (rc 2) rather than run an expensive job under an incoherent profile. A true
no-op
102         (returns True, ZERO work) when no deployment.profile is selected.
103
104         The CUDA probe is gated behind an active profile: ``probe_cuda_available()`` lazily
imports
105         torch, and pre-M5 the worker's infer/train path was torch-free (detection is
delegated to a
106         WSL/native subprocess). Probing only when a profile is set keeps the default-OFF
path a true
107         no-op of work, not just of output."""
108         profile = getattr(getattr(config, "deployment", None), "profile", "") or ""
109         cuda = probe_cuda_available() if profile else None # torch-free on the default-OFF
path
110         try:
111             enforce_startup_checks(config, cuda_available=cuda, logger=logger)
112             return True
113         except StartupCheckError:
114             return False # already logged at ERROR by enforce_startup_checks
115
116
117     def _dispatch_remote(compute: Any, args: argparse.Namespace) -> int:
118         """M6: hand ONE infer job to a remote ComputeBackend (Cloud Run) and return its rc
once the
119         outputs are durably in the bucket (the backend bucket-polls the done-marker). The
dispatched
120         container runs this SAME cmd_infer INLINE (compute_target forced local_gpu) ? never
re-dispatch.
121
122         A remote job that never writes a completion marker (a failed/hung container) makes
the bucket
123         poll exhaust its budget ? ``PolicyTimeout``. Catch it and return a CLEAN rc 4
(remote dispatch
124         timeout/failure, distinct from the local 2/3) rather than letting a raw traceback
escape."""
125         from backend.infrastructure.execution_policy import PolicyTimeout
126
127         spec = ComputeJobSpec(
128             video_uri=args.video_uri, out_uri=args.out_uri,
129             segmenter=args.segmenter, config_overrides=tuple(args.set or ()),
130         )
131         try:
132             result = compute.run_infer(spec)
133         except PolicyTimeout as e:
134             logger.warning("[worker] remote compute did not complete (no marker within
budget): %s", e)
135             return 4
136         logger.info("[worker] remote compute done: video_id=%s rc=%d outputs=%s",
137                     result.video_id, result.returncode, result.outputs_uri)
138         return result.returncode
139
140
141     def _write_done_marker(done_uri: str, video_id: str, returncode: int, segment_count:
int,
142                           status: str, storage_cfg: dict, scratch: str) -> None:
143         """M6: write a tiny JSON completion marker to ``done_uri`` (via the storage layer)
for a remote
144         dispatcher's bucket-poll. Best-effort ? never raises into the worker's success
return."""
145         try:
146             marker = {"video_id": video_id, "returncode": returncode,
147                      "segment_count": segment_count, "status": status}
148             local = os.path.join(scratch, "_done.json")
149             with open(local, "w", encoding="utf-8") as f:

```

```

150         json.dump(marker, f)
151         storage.put(local, done_uri, config=storage_cfg)
152         logger.info("[worker] wrote completion marker -> %s", done_uri)
153     except Exception as e: # never fail a good run because the marker push hiccuped
154         logger.warning("[worker] could not write done-marker %s: %s", done_uri, e)
155
156
157     def cmd_infer(args: argparse.Namespace) -> int:
158         config = load_config(args.config) if args.config else load_config()
159         _apply_overrides(config, args.set)
160         if not _run_startup_checks(config):
161             return 2
162
163         # M6: a cloud compute_target DISPATCHES to a Cloud Run Job instead of running in-
164         # process.
165         # DEFAULT-OFF: get_compute_backend returns None for ""/local_gpu/cpu_mock ? fall
166         # through to the
167         # unchanged inline path below. The dispatched container runs THIS cmd_infer inline
168         # (the
169         # dispatcher forces compute_target=local_gpu), so it never recursively re-
170         # dispatches.
171         compute = get_compute_backend(config)
172         if compute is not None:
173             return _dispatch_remote(compute, args)
174
175         storage_cfg = config.storage.model_dump()
176
177         scratch = args.scratch or tempfile.mkdtemp(prefix="rally-worker-")
178         os.makedirs(scratch, exist_ok=True)
179         config.output.output_dir = scratch
180
181         # 1. PULL ? local:// / bare path = identity passthrough; s3:// / gcs:// = download
182         # to scratch.
183         local_video = storage.fetch(args.video_uri, os.path.join(scratch, "in.mp4"),
184         config=storage_cfg)
185         print(f"[worker] pulled {args.video_uri} -> {local_video}")
186
187         # 2. IDENTITY ? whole-file MD5 (the labels?video join key; pulled bytes must be
188         # byte-identical).
189         video_id = compute_video_id(local_video)
190
191         # 3. VALIDATE (same registry as the main CLI).
192         val_results, val_context = validator_registry.run_all_with_context(local_video,
193         config)
194         failures = [r for r in val_results if not r.passed]
195         db = Database(os.path.join(scratch, config.output.db_name))
196         db.add_video(video_id, local_video, "failed" if failures else "processed",
197         [r.model_dump() for r in val_results])
198
199         segments: List[dict] = []
200         segmenter_actual = None
201         segmentation_ran = False
202         status = "failed"
203         try:
204             if failures:
205                 print("[worker] validation FAILED: "
206                 + "; ".join(f"{r.validator_name}: {r.message}" for r in failures))
207                 return 2
208             seg_name = args.segmenter or config.indexing.default_segmenter
209             segmenter_cls = segmenter_registry.get(seg_name)
210             if not segmenter_cls:
211                 print(f"[worker] unknown segmenter: {seg_name!r} "
212                 f"(have {list(segmenter_registry.list_available())})")
213                 return 2
214             segmenter_actual = seg_name

```

```

207         result = segmenter_cls(db, config).process_video(video_id, local_video,
max_frames=args.max_frames)
208         segmentation_ran = True
209         if isinstance(result, Failure):
210             print(f"[worker] segmenter FAILED: {result.message}")
211             return 3
212         segments = result.value if hasattr(result, "value") else result
213         status = "success"
214         # Loud, never-silent diagnostic: a 0-segment success is almost always a
misconfig, not an
215         # empty match ? the cold-start failure mode (the substrate detector yielded no
usable
216         # trajectories). Explain it so a run is analysable from the log alone (re-run -v
for more).
217         if not segments:
218             detector_impl = (os.environ.get("RALLY_DETECTOR_IMPL")
219                             or getattr(config.indexing, "detector_impl", None) or
"auto")
220             logger.warning(
221                 "infer produced 0 segments for video_id=%s (segmenter=%s,
detector_impl=%s). "
222                 "The detector substrate yielded no usable trajectories/windows ? check
the detector "
223                 "logs above; common causes: native runner UNAVAILABLE
(weights/repo/WASB_PYTHON env), "
224                 "the WASB env produced no detections, or the input resolution differs
from the tuned "
225                 "proxy resolution. Re-run with -v for the native command + frame
counts.",
226                 video_id, segmenter_actual, detector_impl,
227             )
228         finally:
229             # Best-effort per-video observability report (never raises) ? parity with the
main CLI.
230             emit_indexer_report(
231                 db=db, video_id=video_id, video_path=local_video, config=config,
232                 val_results=val_results, val_context=val_context,
233                 segmenter_requested=args.segmenter, segmenter_actual=segmenter_actual,
234                 was_reingest=False, segmentation_ran=segmentation_ran,
235             )
236
237         # 5. SERIALIZE + 6. PUSH (outputs/<video_id>/?). idempotent on video_id.
238         out_local = os.path.join(scratch, "outputs", video_id)
239         os.makedirs(out_local, exist_ok=True)
240         _write_intervals_csv(segments, os.path.join(out_local, "intervals.csv"))
241         _write_report_json(video_id, segments, status, os.path.join(out_local,
"report.json"))
242
243         out_prefix = args.out_uri.rstrip("/")
244         pushed = []
245         for fn in sorted(os.listdir(out_local)):
246             uri = f"{out_prefix}/outputs/{video_id}/{fn}"
247             storage.put(os.path.join(out_local, fn), uri, config=storage_cfg)
248             pushed.append(uri)
249
250         print(f"[worker] infer done: {len(segments)} segment(s); pushed {len(pushed)}
artifact(s):")
251         for u in pushed:
252             print("    ", u)
253
254         # M6: a remote dispatcher passes --done-uri; write a completion marker so its
bucket-poll
255         # terminates (carries video_id ? the dispatcher's output path + the worker rc).
DEFAULT-OFF:
256         # no --done-uri ? no marker (unchanged local run). Written on SUCCESS only ? a

```

```

failed container
257     # job currently lets the dispatcher's bounded poll time out (a failure-marker is a
follow-up).
258     if getattr(args, "done_uri", None):
259         _write_done_marker(args.done_uri, video_id, 0, len(segments), status,
storage_cfg, scratch)
260     return 0
261
262
263     #: Video extensions accepted under <corpus>/videos/ (mirrors the labels/ '.csv' filter
so a
264     #: per-video sidecar ? .srt/.json/thumbnail ? can't shadow the real video on a stem
collision).
265     _VIDEO_EXTS = (".mp4", ".mov", ".mkv", ".avi", ".webm", ".m4v")
266
267
268     def _probe_video_fps_width(path: str):
269         """Robust ffprobe (fps, frame_width) for a real video. Returns None on ANY failure.
270
271         The detector trajectory is frame-indexed; the harness maps frame->time via fps, so a
WRONG
272         fps silently mis-aligns every rally label (e.g. a 60fps clip read as 30 doubles
every time).
273         cv2's ``_probe_fps_width`` silently defaults to (30, 1280) on a read miss ? fine for
the
274         inference report (it flags the fallback) but corrupting for REAL training data. So
here we
275         probe with ffprobe (same tool as ``get_video_duration``) and return None so the
caller SKIPS
276         rather than guesses.
277         """
278         import json
279         import subprocess
280         try:
281             out = subprocess.check_output(
282                 [ffprobe_bin(), "-v", "error", "-select_streams", "v:0",
283                  "-show_entries", "stream=r_frame_rate,width", "-of", "json", path],
284                 stderr=subprocess.DEVNULL).decode()
285             st = (json.loads(out).get("streams") or [{}])[0]
286             num, _, den = str(st.get("r_frame_rate", "0/1")).partition("/")
287             fps = float(num) / float(den or "1")
288             width = float(st.get("width") or 0)
289             if fps > 0 and width > 0:
290                 return fps, width
291         except (subprocess.SubprocessError, ValueError, KeyError, OSError,
json.JSONDecodeError):
292             pass
293         return None
294
295
296     def _resolve_train_detector(args: argparse.Namespace, config: Any):
297         """Build the trajectory DetectorRunner for ``--trajectory-source detector``, honouring
298         detector_impl (RALLY_DETECTOR_IMPL / indexing.detector_impl: auto=WSL, native=GPU
box,
299         stub=CPU mock). Returns the runner, or prints an error + returns None.
300
301         Reuses the segmenter's ``_resolve_runner`` seam so the worker and the live CLI build
the
302         detector identically (one source of truth). A non-trajectory segmenter (e.g. yolo)
has
303         no shuttle detector and is rejected.
304         """
305         from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
306
307         seg_cls = segmenter_registry.get(args.segmenter)

```



```

308         if not seg_cls:
309             print(f"[worker] unknown segmenter: {args.segmenter!r} "
310                   f"(have {list(segmenter_registry.list_available())})")
311             return None
312         seg = seg_cls(None, config)
313         if not isinstance(seg, TrajectoryHybridSegmenter):
314             print(f"[worker] --segmenter {args.segmenter!r} has no shuttle detector; "
315                   f"use a trajectory hybrid (wasb_hybrid / tracknet_hybrid /
fusion_hybrid).")
316             return None
317         try:
318             runner, _ = seg._resolve_runner(config.get("indexing", {}))
319         except NotImplementedError as e:
320             # e.g. detector_impl='native' for a family without a native runner (tracknet, or
321             # fusion with a tracknet substrate). Fail cleanly (rc!=0), not with a raw
traceback.
322             print(f"[worker] detector not available: {e}")
323             return None
324         ok, msg = runner.healthcheck()
325         if not ok:
326             print(f"[worker] detector environment not ready: {msg}")
327             return None
328         print(f"[worker] detector ready ({args.segmenter}): {msg}")
329         return runner
330
331
332     def cmd_train(args: argparse.Namespace) -> int:
333         """Gen-0 train-from-bucket: pull labels (+ videos) -> trajectories -> manifest ->
shell out
334         to training.gen0.harness (backend must NOT import training) -> push the artifact
bundle.
335
336         Two trajectory sources (the train pipeline + harness are identical either way):
337         * ``synth`` (default) ? the CPU "mock GPU": deterministic, label-consistent
synthetic
338             shuttle paths (no GPU/WSL), so the whole pipeline is validated on a CPU box /
CI.
339         * ``detector`` ? REAL shuttle trajectories: pull each video and run the resolved
340             DetectorRunner (native WASB on a GPU box, or stub for a GPU-free wiring test).
This
341             is the M3.5 "first real training" path; only the trajectory source flips.
342         """
343         import subprocess
344
345         config = load_config(args.config) if args.config else load_config()
346         _apply_overrides(config, args.set)
347         if not _run_startup_checks(config):
348             return 2
349         storage_cfg = config.storage.model_dump()
350
351         scratch = args.scratch or tempfile.mkdtemp(prefix="rally-train-")
352         labels_dir = os.path.join(scratch, "labels")
353         traj_dir = os.path.join(scratch, "trajectories")
354         videos_dir = os.path.join(scratch, "videos")
355         os.makedirs(labels_dir, exist_ok=True)
356         os.makedirs(traj_dir, exist_ok=True)
357
358         corpus = args.corpus_uri.rstrip("/")
359         label_uris = sorted(u for u in storage.list_uris(f"{corpus}/labels/",
config=storage_cfg)
360                             if u.lower().endswith(".csv"))
361         if len(label_uris) < 2:
362             print(f"[worker] need >=2 labeled videos for leave-one-video-out; found
{len(label_uris)} "
363                   f"under {corpus}/labels/")

```

```

364         return 2
365
366     # Real-detector source: resolve the runner once + index the bucket's videos/ by
stem.
367     runner = None
368     video_by_stem: dict = {}
369     if args.trajectory_source == "detector":
370         os.makedirs(videos_dir, exist_ok=True)
371         runner = _resolve_train_detector(args, config)
372         if runner is None:
373             return 2
374         for vuri in storage.list_uris(f"{corpus}/videos/", config=storage_cfg):
375             vname = storage.parse_uri(vuri).name
376             if not vname.lower().endswith(_VIDEO_EXTS):
377                 continue # skip sidecars (.srt/.json/thumbnails) so they can't shadow
the video
378                 video_by_stem[os.path.splitext(vname)[0]] = vuri
379
380     print(f"[worker] trajectory source: {args.trajectory_source}")
381     specs: List[dict] = []
382     for uri in label_uris:
383         fname = storage.parse_uri(uri).name # e.g.
GX020094_proxy.rallies.csv
384         name = fname.replace(".rallies.csv", "").replace(".csv", "")
385         local_label = storage.fetch(uri, os.path.join(labels_dir, fname),
config=storage_cfg)
386         if args.trajectory_source == "detector":
387             vuri = video_by_stem.get(name)
388             if not vuri:
389                 print(f"[worker] no video for {name!r} under {corpus}/videos/ ?
skipping.")
390                 continue
391             local_video = storage.fetch(vuri, os.path.join(videos_dir,
storage.parse_uri(vuri).name),
392                                     config=storage_cfg)
393             video_id = compute_video_id(local_video)
394             # Real fps/width drive the trajectory frame->time mapping; PROBE (don't
guess) ? a
395             # wrong fps silently mis-aligns every label, so skip the video if ffprobe
can't read it.
396             meta = _probe_video_fps_width(local_video)
397             if meta is None:
398                 print(f"[worker] could not probe fps/width for {name!r} (ffprobe) ?
skipping (won't guess).")
399                 continue
400             fps, frame_width = meta
401             traj = runner.run_predict(local_video, traj_dir, video_id=video_id)
402             if not traj:
403                 print(f"[worker] detector produced no trajectory for {name!r} ?
skipping.")
404                 continue
405             specs.append({"name": name, "trajectory": traj, "labels": local_label,
406                          "fps": fps, "frame_width": frame_width})
407         else:
408             from backend.pipeline.detectors.stub_runner import
synth_trajectory_from_labels
409             traj = os.path.join(traj_dir, f"{name}_traj.csv")
410             synth_trajectory_from_labels(local_label, traj, fps=args.fps,
frame_width=args.frame_width)
411             specs.append({"name": name, "trajectory": traj, "labels": local_label,
412                          "fps": args.fps, "frame_width": args.frame_width})
413
414     if len(specs) < 2:
415         print(f"[worker] need >=2 videos with trajectories for leave-one-video-out; got
{len(specs)}.")

```

```

416         return 2
417     manifest_path = os.path.join(scratch, "manifest.json")
418     with open(manifest_path, "w", encoding="utf-8") as f:
419         json.dump(specs, f, indent=2)
420     src_label = "real detector" if args.trajectory_source == "detector" else "CPU-mock
stub"
421     print(f"[worker] built manifest: {len(specs)} videos ({src_label} trajectories)")
422
423     out_dir = os.path.join(scratch, "artifacts")
424     cmd = [sys.executable, "-m", "training.gen0.harness",
425           "--manifest", manifest_path, "--out", out_dir, "--version", args.version]
426     if args.floor:
427         floor_local = (storage.fetch(args.floor, os.path.join(scratch, "floor.json"),
config=storage_cfg)
428                       if "://" in args.floor else args.floor)
429         cmd += ["--floor", floor_local]
430     print("[worker] training:", " ".join(cmd))
431     rc = subprocess.run(cmd).returncode
432     if rc != 0:
433         print(f"[worker] gen0 harness failed (rc={rc})")
434         return rc
435
436     # PUSH the versioned artifact bundle (weights.json + manifest.json) to the bucket.
437     bundle = os.path.join(out_dir, args.version)
438     out_prefix = args.out_uri.rstrip("/")
439     pushed = []
440     for fn in sorted(os.listdir(bundle)):
441         uri = f"{out_prefix}/weights/{args.version}/{fn}"
442         storage.put(os.path.join(bundle, fn), uri, config=storage_cfg)
443         pushed.append(uri)
444     print(f"[worker] train done: pushed {len(pushed)} artifact(s):")
445     for u in pushed:
446         print("    ", u)
447     return 0
448
449
450 def build_parser() -> argparse.ArgumentParser:
451     p = argparse.ArgumentParser(prog="python -m backend.worker",
452                                description="Storage-aware worker: pull ? run pipeline ?
push.")
453     p.add_argument("-v", "--verbose", action="store_true",
454                   help="DEBUG logging (the native detector command, frame counts, per-
stage detail)")
455     sub = p.add_subparsers(dest="cmd", required=True)
456
457     pi = sub.add_parser("infer", help="run inference on one video URI and push outputs")
458     pi.add_argument("--video-uri", required=True, help="local path / local:// / s3:// /
gcs://")
459     pi.add_argument("--out-uri", required=True, help="output prefix
(outputs/<video_id>/? is appended)")
460     pi.add_argument("--segmenter", default=None, help="default:
indexing.default_segmenter")
461     pi.add_argument("--config", default=None, help="config.json path (default:
./config.json)")
462     pi.add_argument("--scratch", default=None, help="local scratch dir (default: a temp
dir)")
463     pi.add_argument("--max-frames", type=int, default=None)
464     pi.add_argument("--done-uri", default=None,
465                     help="M6: storage URI for a completion MARKER (video_id + rc)
written on a "
466                     "successful run, so a remote (Cloud Run) dispatcher's bucket-
poll terminates. "
467                     "Default-OFF: omit it for the normal local run.")
468     pi.add_argument("--set", action="append", default=[], metavar="k=v",
469                     help="config override, e.g. --set indexing.skip_ai_handoff=true")

```

```

470     pi.set_defaults(func=cmd_infer)
471
472     pt = sub.add_parser("train", help="Gen-0 train-from-bucket (labels [+videos] ->
trajectories -> harness)")
473     pt.add_argument("--corpus-uri", required=True, help="prefix containing labels/ (+
videos/ for "
474                     "--trajectory-source detector), e.g. s3://bucket or gcs://bucket")
475     pt.add_argument("--out-uri", required=True, help="output prefix (weights/<version>/
is appended)")
476     pt.add_argument("--version", required=True, help="semver for the artifact bundle")
477     pt.add_argument("--trajectory-source", choices=["synth", "detector"],
default="synth",
478                     help="synth = CPU-mock label-consistent trajectories (default, no
GPU); "
479                     "detector = REAL shuttle trajectories via the resolved
DetectorRunner "
480                     "(needs videos/ in the corpus; honours detector_impl).")
481     pt.add_argument("--segmenter", default="wasb_hybrid",
482                     help="trajectory hybrid whose detector is used for --trajectory-
source detector")
483     pt.add_argument("--floor", default=None, help="floor baseline JSON (local path or
storage URI)")
484     pt.add_argument("--config", default=None)
485     pt.add_argument("--scratch", default=None)
486     pt.add_argument("--fps", type=float, default=30.0, help="synth trajectory fps
(detector probes the video)")
487     pt.add_argument("--frame-width", type=float, default=1280.0,
488                     help="synth trajectory frame width (detector probes the video)")
489     pt.add_argument("--set", action="append", default=[], metavar="k=v")
490     pt.set_defaults(func=cmd_train)
491     return p
492
493
494     def _setup_logging(verbose: bool) -> None:
495         """Configure logging for the worker CLI so the pipeline's existing INFO/ERROR
diagnostics are
496         VISIBLE ? the worker previously ran with no logging config, so every
segmenter/detector log
497         (detector choice, 'Running native WASB inference', availability failures) was
silently dropped.
498         That is exactly why a 0-rally cold-start gave nothing to analyse. Configured here
(the CLI
499         entry), not at import, so importing the module never reconfigures a caller's
logging."""
500         # force=True: basicConfig is a no-op if the root already has handlers (e.g. a
library
501         # configured logging at import), which would leave the level at WARNING and re-
suppress the
502         # diagnostics. Forcing reconfiguration means the worker reliably controls its own
verbosity.
503         logging.basicConfig(
504             level=logging.DEBUG if verbose else logging.INFO,
505             format="%(asctime)s [%(levelname)s] %(name)s: %(message)s",
506             datefmt="%H:%M:%S",
507             force=True,
508         )
509
510
511     def main(argv=None) -> int:
512         args = build_parser().parse_args(argv)
513         _setup_logging(getattr(args, "verbose", False))
514         return args.func(args)
515
516
517     if __name__ == "__main__":

```

```
518         sys.exit(main())
519
```

10. assistant (2026-06-22T10:25:39.221Z)

Now let me look at the storage ref.py and check how eval/litmus currently reads paths, plus how the manifest naming interacts with train's stem matching.

11. user (2026-06-22T10:25:40.186Z)

```
1      """Storage value objects + URI parsing for the pluggable storage layer.
2
3      A ``StorageRef`` is a backend-agnostic pointer to one object. URIs:
4
5          <bare path>                -> local    (today's behaviour; e.g. C:/v/x.mp4,
/data/x.mp4)
6          local://<path>             -> local    (local:///abs or local://rel/p)
7          s3://<bucket>/<key>        -> s3       (AWS + R2 + D0 Spaces + MinIO via
endpoint_url)
8          gcs://<bucket>/<key>      -> gcs      (gs:// accepted, normalized to gcs://)
9          gdrive://<path>           -> gdrive   (path under your locally-MOUNTED Google
Drive "My Drive")
10
11      No cloud SDKs are imported here ? this module is pure stdlib so ``import
backend.storage``
12      stays cheap.
13      """
14      from __future__ import annotations
15
16      import re
17      from dataclasses import dataclass
18      from datetime import datetime
19      from typing import Optional
20
21      _SCHEME_RE = re.compile(r"^[A-Za-z][A-Za-z0-9+.\-]*://")
22      _KNOWN = ("local", "s3", "gcs", "gdrive")
23
24
25      @dataclass(frozen=True)
26      class StorageStat:
27          """Lightweight object metadata (uniform across backends)."""
28          size: int
29          modified: Optional[datetime] = None
30          etag: Optional[str] = None
31          content_type: Optional[str] = None
32
33
34      @dataclass(frozen=True)
35      class StorageRef:
36          """A backend + a location. ``bucket`` is the S3/GCS bucket; for ``local`` and
``gdrive`` it is
37          empty and ``key`` carries the path (a filesystem path, or a path under the mounted
Drive)."""
38          backend: str
39          bucket: str
40          key: str
41          uri: str
42
43          @property
44          def name(self) -> str:
45              return self.key.rstrip("/").rsplit("/", 1)[-1]
46
47
48      def parse_uri(uri: str) -> StorageRef:
49          """Parse a storage URI (or a bare filesystem path) into a ``StorageRef``.
```

```

50
51     A string with no ``scheme://`` prefix (including Windows ``C:\\...`` paths, which
52     have no ``//``) is treated as a local filesystem path ? preserving today's
behaviour.
53     """
54     s = str(uri)
55     m = _SCHEME_RE.match(s)
56     if not m:
57         return StorageRef("local", "", s, f"local://{s}")
58     scheme = m.group(1).lower()
59     rest = s[m.end():]
60     if scheme == "gs":
61         scheme = "gcs"
62     if scheme in ("local", "gdrive"):
63         # Path-keyed, no bucket: local FS, or a path under the mounted Google Drive "My
Drive".
64         return StorageRef(scheme, "", rest, s)
65     if scheme in _KNOWN:
66         bucket, _, key = rest.partition("/")
67         return StorageRef(scheme, bucket, key, s)
68     raise ValueError(f"unsupported storage URI scheme: {scheme}://{s} (in {s!r})")
69
70
71     def resolve_input_uri(raw: str, *, input_backend: str = "local", input_root: str = "")
-> str:
72         """Resolve a user-supplied input reference into a storage URI
(COMPUTE_DECOUPLED_SERVING M2).
73
74         Precedence (first match wins):
75         1. An explicit ``scheme://`` on ``raw`` (``gs://``, ``s3://``, ``local://``,
``gdrive://``) ? as-is.
76         2. An ABSOLUTE local path (``/data/x.mp4``, ``C:\\?``, ``C:/?``) ? as-is (always
local).
77         3. A bare/relative name + a non-empty ``input_root`` ? joined under it
(``gs://bucket/videos`` + ``x.mp4`` ? ``gs://bucket/videos/x.mp4``).
78         4. A bare/relative name + ``input_backend != "local"`` ? prefixed (``gcs`` +
``bucket/x`` ?
80         ``gcs://bucket/x``).
81         5. Otherwise ? ``raw`` unchanged (a relative/bare LOCAL path ? today's behaviour).
82
83         So the default (``input_backend="local"`` , ``input_root=""``) is an identity
passthrough for every
84         path used today, preserving byte-for-byte behaviour.
85         """
86         s = str(raw)
87         if _SCHEME_RE.match(s):          # 1. explicit scheme wins
88             return s
89         # 2. an anchored LOCAL path is always local, never joined. Detect this OS-
INDEPENDENTLY ?
90         #     os.path.isabs is platform- AND Python-version-dependent (on Windows + py3.13 a
POSIX-style
91         #     "/data/x" is NOT absolute). Anchored == a leading "/" or "\\", or a Windows
drive prefix "X:".
92         if s.startswith(("/", "\\")) or (len(s) >= 2 and s[1] == ":" and s[0].isalpha()):
93             return s
94         if input_root:                    # 3. join a relative name under the configured
root
95             return input_root.rstrip("/") + "/" + s.lstrip("/")
96         if input_backend and input_backend != "local":    # 4. prefix a bucket/key with the
declared backend
97             return f"{input_backend}:{s}"
98         return s                                # 5. bare/relative local path ? unchanged
99

```

Found 41 files

- docs\DATA_IN_GCS.md
- tests\test_config.py
- docs\NEXT_STEPS.md
- docs\COMPUTE_DECOUPLED_SERVING\README.md
- docs\CODE_MAP.md
- backend\config\models.py
- backend\config\presets.py
- tests\test_input_source.py
- backend\storage__init__.py
- backend\storage\ref.py
- docs\data_pipeline\GOLDEN_DATA_SHARING.md
- docs\RALLY_DETECTION_QUALITY_REPORT.md
- docs\POST_166_WORK_ITEMS_MILESTONES_AND_RISK_ASSESSMENT.md
- docs\PROMINENT_COURT_DETECTION.md
- docs\GOLDEN_REGRESSION_FIXTURES.md
- docs\ABLATION_LAYER.md
- docs\data_pipeline\GOLDEN_VIDEOS.md
- docs\QUALITY_ITERATIONS.md
- docs\COMPUTE_DECOUPLED_SERVING\TESTING_STRATEGY.md
- training\gen0\harness.py
- backend\pipeline\segmenters\fusion_hybrid.py
- backend\eval\rally_seg_eval.py
- eval_baselines\nightly_baseline.json
- tests\test_ablation.py
- backend\eval\fusion_audio.py
- backend\eval\ablation.py
- tests\test_served_gate_a_regression.py
- scripts\nightly_regression.py
- docs\archives\quality-iterations\2026-06-18-lovo-resweep-n13\lovo_resweep.py
- scripts\nightly_regression.ps1
- docs\archives\CODE_AUDIT_AND_TEST_HARDENING-COMPLETED-2026-06-14.md
- docs\archives\HARDENING_LOOP_HANDOFF-COMPLETED-2026-06-14.md
- backend\eval\serve_contrast.py
- eval_baselines\experiment_leaderboard.json
- backend\eval\served_gate_a.py
- backend\eval\fusion_golden.py
- docs\archives\quality-iterations\2026-06-first-ab-experiment\README.md
- backend\eval\rally_seq_proto.py
- eval_baselines\heuristic_lovo_n6.json
- artifacts\rally_tal_gen0\0.1.0\manifest.json
- docs\archives\quality-iterations\2026-06-multivideo-lovo\README.md

13. assistant (2026-06-22T10:25:50.089Z)

Let me examine the served Gate-A litmus test and eval paths to confirm they're pinned to local paths, and check the manifest naming-vs-stem matching issue.

14. user (2026-06-22T10:25:51.052Z)

```

1      """SERVED Gate-A regression litmus ? the *production* FusionSegmenter path on real
footage.
2
3      The flagged gap (#146): the served default flip (`default_segmenter=fusion_hybrid` +
4      ``min_crossings=2``) is **mock-tested only** on real video (`test_fusion_hybrid.py`
drives a
5      MagicMock runner). The offline ablation litmus
6      (`test_ablation.test_litmus_reproduces_gate_a`)
7      validates the Gate-A *signal* ? but on the **permissive proposal windowing**
8      (`distill_local.PROPOSAL`), NOT the production operating point. This module closes the
gap: it
9      runs the real ``FusionSegmenter`` decision seams (`enrich_candidate_stats` net-
crossings +
10     ``select_for_handoff`` Gate A) over windows from the **production**
11     ``trajectory_to_action_windows`` config, on the cached golden WASB trajectories, and

```

```

asserts
11     the served path's measured behaviour.
12
13     GPU-free / Gemini-free (person cue OFF) ? it stops at the AI-handoff boundary. Skips
cleanly in
14     CI where the golden trajectories are absent; runs on the owner box where they are
present, over
15     **whatever golden clips are currently on disk** (`_golden_specs` requires >= 6).
16
17     CORPUS NOTE ? re-grounded 2026-06-18 (the golden set is mutable and grows over time).
The
18     original 6 *single-court* golden clips this litmus was first calibrated on (2026-06-14)
have been
19     retired from the box ? their labels are gone ? so the present golden set is the 9
**multicourt**
20     clips now in ``output/`` (mahadevpura_1/2, GX0101xx, GX0x0094, Badminton_BXH_2,
Boxhill_Doubles).
21     On that corpus the served path is **recall-starved**: WASB tracks a single shuttle while
several
22     courts rally at once, so ``trajectory_to_action_windows`` emits FEWER windows than there
are
23     rallies (served over-seg ratio < 1, i.e. under-segmentation). Because Gate A only ever
DROPS
24     windows, on this under-segmented corpus it is mildly **F1-negative** (mean served ?F1 ?
-0.034),
25     not the ~neutral -0.008 the original 6 showed. Under the permissive PROPOSAL windowing
the same
26     gate is ~F1-neutral (?F1 ? -0.001) while still cutting over-segmentation (1.16?0.64).
27
28     The canonical **+0.074 / 6-of-6 / p=0.031** Gate-A win was a property of the
*original-6* corpus
29     and is NOT reproduced here (those clips are no longer on the box); it is preserved in
the archive
30     (`docs/archives/quality-iterations/2026-06-first-ab-experiment/``). These tests now pin
the
31     **present-corpus** behaviour so a future windowing/corpus change that silently shifts it
trips the
32     litmus. The pinned magnitudes are a date-stamped snapshot of the present golden set and
will need
33     re-measurement as the corpus grows.
34     ""
35     from __future__ import annotations
36
37     import json
38     import os
39
40     import pytest
41
42     # cv2 is imported transitively by trajectory_hybrid (the served engine). On a box
without it
43     # (CI), skip the whole module rather than erroring at import.
44     pytest.importorskip("cv2")
45
46     from backend.eval import served_gate_a as sga # noqa: E402
47     from backend.pipeline.detectors.tracknet_runner import TrackNetRunner # noqa: E402
48     from backend.pipeline.detectors import rally_gate # noqa: E402
49     from backend.eval.calibrate_wasb import load_golden_gts # noqa: E402
50     from backend.eval.metrics import score # noqa: E402
51     from backend.eval import segmentation_metrics as _segm # noqa: E402
52
53     # The golden manifest lives in the MAIN checkout's output/ (gitignored; absent in a
fresh
54     # worktree / CI). Resolve it relative to this repo root first, with an env override for
an
55     # out-of-tree checkout.

```



```

56     _REPO_ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
57     MANIFEST = os.environ.get(
58         "SERVED_GATE_A_MANIFEST",
59         os.path.join(_REPO_ROOT, "output", "human_lovo_manifest.json"),
60     )

```

15. user (2026-06-22T10:25:51.742Z)

```

1     """SERVED Gate-A verification: run the *production* ``FusionSegmenter`` decision seams
2     on a cached WASB trajectory and score the resulting AI-handoff windows vs golden labels.
3
4     The offline litmus (``backend.eval.ablation`` / ``fusion_compare``) measures Gate A on
5     the
6     **permissive proposal windowing** (``distill_local.PROPOSAL`` = recall-oriented) re-
7     scored
8     through the experiment harness's ``_gate_mask``. That validates the *signal*. It does
9     NOT
10    exercise the **served path**: in production the windows come from
11    ``trajectory_to_action_windows`` at the production operating point
12    (``velocity_threshold=0.02, dead_time_merge_gap=2.0, min_action_window_duration=2.0``),
13    the
14    net-crossing count is computed by ``FusionSegmenter._enrich_candidate_stats``, and the
15    gate is
16    ``FusionSegmenter._select_for_handoff``. THIS module drives exactly those production
17    methods ?
18    no reimplementations of the windowing or the gate ? on a cached trajectory CSV, so the
19    verification reflects what ``process_video`` actually hands to Gemini.
20
21    GPU-free and Gemini-free: it stops at the handoff-selection boundary (the windows that
22    WOULD
23    be sent to the AI). With the person cue OFF (the default) nothing here loads torch or
24    touches
25    the GPU ? it just re-scores cached WASB trajectories.
26
27    Run (owner box, golden trajectories present):
28    python -m backend.eval.served_gate_a --manifest output/human_lovo_manifest.json
29
30    """
31    from __future__ import annotations
32
33    import argparse
34    import json
35    from dataclasses import dataclass
36    from typing import Any, Dict, List, Optional, Tuple
37
38    from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
39    from backend.pipeline.segmenters.fusion_hybrid import FusionSegmenter
40    from backend.eval.calibrate_wasb import load_golden_gts
41    from backend.eval.metrics import Interval, score, Score
42    from backend.eval import segmentation_metrics as _segm
43
44    # Production windowing operating point (config.json indexing defaults ? the values
45    # TrajectoryHybridSegmenter.process_video reads for the served fusion run).
46    PROD_VELOCITY_THRESH = 0.02
47    PROD_MERGE_GAP = 2.0
48    PROD_MIN_WINDOW = 2.0
49
50
51    def _served_idx_cfg(min_crossings: int) -> Dict[str, Any]:
52        """The ``indexing`` config dict the served FusionSegmenter seams read ? mirrors
53        config.json's ``indexing.fusion`` block, with Gate A at ``min_crossings`` and the
54        person cue OFF (so no GPU / torch is touched)."""
55        return {
56            "fusion": {
57                "substrate": "wasb",
58                "min_crossings": min_crossings,

```

```

50         "min_players": 0,
51         "cue_flags": {},
52     },
53     "wasb": {"velocity_threshold": PROD_VELOCITY_THRESH},
54     "dead_time_merge_gap": PROD_MERGE_GAP,
55     "min_action_window_duration": PROD_MIN_WINDOW,
56 }
57
58
59 def served_handoff_windows(trajecory_csv: str, fps: float, frame_width: float,
60                           min_crossings: int) -> List[Interval]:

```

16. assistant (2026-06-22T10:25:56.164Z)

This confirms the litmus is pinned to a local `output/human\_lovo\_manifest.json` with C:-absolute paths inside it. Let me check the manifest structure (what paths it points to) and how `load\_golden\_gts` reads the labels, plus check the manifest naming issue with the train stem-matching. Let me also check how the GCS manifest entries are shaped vs what the litmus/eval expects.

17. user (2026-06-22T10:25:57.290Z)

```

60         min_crossings: int) -> List[Interval]:
61     """Run the PRODUCTION FusionSegmenter decision path on a cached trajectory and
62     return the
63     windows that would reach the AI handoff (i.e. the served rally candidates).
64     Uses the real ``FusionSegmenter`` seam methods ? ``_enrich_candidate_stats``
65     (computes
66     ``net_crossings``) and ``_select_for_handoff`` (Gate A) ? over windows from the
67     production
68     ``trajectory_to_action_windows`` operating point. No GPU, no Gemini: the person cue
69     is OFF.
70     """
71     points = TrackNetRunner.parse_trajectory_csv(trajectory_csv)
72     windows = TrackNetRunner.trajectory_to_action_windows(
73         points, fps=fps, frame_width=frame_width,
74         velocity_thresh=PROD_VELOCITY_THRESH, merge_gap=PROD_MERGE_GAP,
75         min_window_duration=PROD_MIN_WINDOW,
76     )
77     idx_cfg = _served_idx_cfg(min_crossings)
78     # Construct the real served segmenter (db=None, config carries the indexing block).
79     We only
80     # call the pure decision seams ? no process_video, no DB, no provider ? exactly as
81     the
82     # registry constructs it for `name`/`description`.
83     seg = FusionSegmenter(None, {"indexing": idx_cfg, "sport": "badminton"})
84     # The served per-window stats (incl. net_crossings) via the production enrichment
85     hook.
86     window_stats = [
87         seg._enrich_candidate_stats(cw, points, fps, frame_width, video_path="",
88                                   idx_cfg=idx_cfg)
89         for cw in windows
90     ]
91     keep = seg._select_for_handoff(windows, window_stats, idx_cfg)
92     return [(windows[i]["start"], windows[i]["end"]) for i in keep]
93
94
95 @dataclass
96 class ServedVideoResult:
97     name: str
98     num_gt: int
99     on: Score # Gate-A ON (min_crossings=2)
100    off: Score # Gate-A OFF (min_crossings=0, control)
101    seg_ratio_on: float # |preds| / |gts| with Gate-A ON (over-segmentation)

```

```

96         seg_ratio_off: float
97         n_handoff_on: int
98         n_handoff_off: int
99
100     @property
101     def delta_f1(self) -> float:
102         return self.on.f1 - self.off.f1
103
104
105     def evaluate_video(spec: Dict[str, Any], iou: float = 0.5,
106                       on_crossings: int = 2) -> ServedVideoResult:
107         """Score the served Gate-A ON (min_crossings=on_crossings) vs OFF (0) handoff
108         windows for
109         one golden video against its golden labels."""
110         fps, fw = float(spec["fps"]), float(spec["frame_width"])
111         traj, labels = str(spec["trajectory"]), str(spec["labels"])
112         gts = load_golden_gts(labels)
113         preds_on = served_handoff_windows(traj, fps, fw, min_crossings=on_crossings)
114         preds_off = served_handoff_windows(traj, fps, fw, min_crossings=0)
115         return ServedVideoResult(
116             name=str(spec["name"]),
117             num_gt=len(gts),
118             on=score(preds_on, gts, iou),
119             off=score(preds_off, gts, iou),
120             seg_ratio_on=_segm.segment_count_ratio(preds_on, gts),
121             seg_ratio_off=_segm.segment_count_ratio(preds_off, gts),
122             n_handoff_on=len(preds_on),
123             n_handoff_off=len(preds_off),
124         )
125
126     def evaluate_manifest(specs: List[Dict[str, Any]], iou: float = 0.5,
127                          on_crossings: int = 2) -> List[ServedVideoResult]:
128         return [evaluate_video(s, iou, on_crossings) for s in specs]
129
130
131     def aggregate(results: List[ServedVideoResult]) -> Dict[str, Any]:
132         """Mean ON/OFF F1 + over-seg ratio, the paired ?F1, and the per-video win count."""
133         n = len(results)
134
135         def _mean(getter) -> float:
136             return sum(getter(r) for r in results) / n if n else 0.0
137
138         deltas = [r.delta_f1 for r in results]
139         wins = sum(1 for d in deltas if d > 1e-9)
140         losses = sum(1 for d in deltas if d < -1e-9)
141         return {
142             "num_videos": n,
143             "mean_f1_on": _mean(lambda r: r.on.f1),
144             "mean_f1_off": _mean(lambda r: r.off.f1),
145             "mean_delta_f1": _mean(lambda r: r.delta_f1),
146             "mean_precision_on": _mean(lambda r: r.on.precision),
147             "mean_precision_off": _mean(lambda r: r.off.precision),
148             "mean_recall_on": _mean(lambda r: r.on.recall),
149             "mean_recall_off": _mean(lambda r: r.off.recall),
150             "mean_seg_ratio_on": _mean(lambda r: r.seg_ratio_on),
151             "mean_seg_ratio_off": _mean(lambda r: r.seg_ratio_off),
152             "wins": wins,
153             "losses": losses,
154         }
155
156
157     def format_table(results: List[ServedVideoResult], agg: Dict[str, Any], iou: float) ->
158     str:
159         lines = [

```

```

159         f"=== SERVED Gate-A verification (production FusionSegmenter path, IoU>={iou})
===",
160         f" {'video':<24s} {'gt':>3s} {'F1_off':>7s} {'F1_on':>7s} {'?F1':>7s} "
161         f"{'segR_off':>9s} {'segR_on':>8s} {'hand_off':>9s} {'hand_on':>8s}",
162     ]
163     for r in results:
164         lines.append(
165             f" {r.name:<24s} {r.num_gt:>3d} {r.off.f1:>7.3f} {r.on.f1:>7.3f} "
166             f"{r.delta_f1:>+7.3f} {r.seg_ratio_off:>9.2f} {r.seg_ratio_on:>8.2f} "
167             f"{r.n_handoff_off:>9d} {r.n_handoff_on:>8d}")
168     lines += [
169         f" {'-' * 92}",
170         f" MEAN F1 {agg['mean_f1_off']:.3f} (off) -> {agg['mean_f1_on']:.3f} (on) "
171         f"?F1={agg['mean_delta_f1']:+.3f} "
172         f"P {agg['mean_precision_off']:.3f}->{agg['mean_precision_on']:.3f} "
173         f"R {agg['mean_recall_off']:.3f}->{agg['mean_recall_on']:.3f}",
174         f" over-seg ratio {agg['mean_seg_ratio_off']:.2f} (off) -> "
175         f"{agg['mean_seg_ratio_on']:.2f} (on) | wins {agg['wins']}/{agg['num_videos']}
"
176         f"(losses {agg['losses']})",
177     ]
178     return "\n".join(lines)
179

```

18. assistant (2026-06-22T10:26:05.961Z)

Key finding: `served\_gate\_a.py` reads `spec["trajectory"]` and `spec["labels"]` as `**plain local file paths**` (`str(spec["trajectory"])`), then TrackNetRunner.parse_trajectory_csv(traj)` / load_golden_gts(labels)` open them directly. It does NOT go through the storage layer (no storage.fetch). So even if manifests/golden.json has gs:// paths, this eval code would try to open() a gs://... string and fail. Let me verify load_golden_gts and parse_trajectory_csv use plain open.`

19. user (2026-06-22T10:26:06.853Z)

```

backend\eval\calibrate_wasb.py:33: def load_golden_gts(csv_path: str) -> List[Interval]:
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\eval\calibrate_wasb.py-34-
    """Rally [start,end] seconds from a golden CSV ? delegates to the canonical loader.
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\eval\calibrate_wasb.py-35-
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\eval\calibrate_wasb.py-36-
    Accepts BOTH golden conventions now (start_time/end_time AND the collector export's
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\eval\calibrate_wasb.py-37-
    start,end ? the historical fork is closed; see gt_loader.py / ADR-008). Keeps this
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\eval\calibrate_wasb.py-38-
    driver's legacy lenient semantics (skip bad rows), but skipped rows are now logged.
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\eval\calibrate_wasb.py-39-
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\eval\calibrate_wasb.py-40-    """
    from
    backend.eval.gt_loader import load_gt_intervals
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\eval\calibrate_wasb.py-41-    return
    load_gt_intervals(csv_path, strict=False)
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\eval\calibrate_wasb.py-42-
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\eval\calibrate_wasb.py-43-
--
backend\pipeline\detectors\tracknet_runner.py:237: def parse_trajectory_csv(csv_win_path:
str) -> List[TrajectoryPoint]:
backend\pipeline\detectors\tracknet_runner.py-238-    """Parse a TrackNet predict CSV
    (columns: Frame,Visibility,X,Y)."""
backend\pipeline\detectors\tracknet_runner.py-239-    points: List[TrajectoryPoint] = []
backend\pipeline\detectors\tracknet_runner.py-240-    with open(csv_win_path, newline="") as
f:
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\detectors\tracknet_runner.py-241-        reader = csv.DictReader(f)
backend\pipeline\detectors\tracknet_runner.py-242-        for row in reader:
backend\pipeline\detectors\tracknet_runner.py-243-            try:
C:\Users\avidu\Projects\badminton-highlight-

```

```

indexer\backend\pipeline\detectors\tracknet_runner.py-244-
points.append(TrajectoryPoint(
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\detectors\tracknet_runner.py-245-
frame=int(row["Frame"]),
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\detectors\tracknet_runner.py-246-
visible=int(row["Visibility"]) == 1,
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\detectors\tracknet_runner.py-247-
x=float(row["X"]),

```

20. assistant (2026-06-22T10:26:16.259Z)

Confirmed. Both use plain `open()` ? they cannot consume `gs://` URIs. The Phase-3 "flip eval/litmus to GCS" is not just a config change; the eval code itself has no storage-layer fetch path. This is a real, concrete gap.

Let me check a couple more things: the train stem-matching against the new `<date>_<name>` label naming (does the date prefix break video matching?), and the `gt_loader` to confirm there's a min-clips floor mismatch.

21. user (2026-06-22T10:26:17.394Z)

```

60     )
61
62
63     def _golden_specs():
64         """Return the manifest specs whose trajectory + label files are all present, else
65         [], """
66         if not os.path.isfile(MANIFEST):
67             return []
68         with open(MANIFEST, encoding="utf-8") as f:
69             specs = json.load(f)
70             ok = [s for s in specs
71                  if os.path.isfile(str(s.get("trajectory", "")))
72                  and os.path.isfile(str(s.get("labels", "")))]
73             return ok if len(ok) >= 6 else []
74
75     _SPECS = _golden_specs()
76     _skip = pytest.mark.skipif(
77         not _SPECS,
78         reason=f"golden trajectories absent (manifest {MANIFEST!r}); runs on the owner box",
79     )
80
81
82     # ----- served-path
83     litmus
84
85     @_skip
86     def test_served_path_runs_on_all_golden_videos():
87         """The production FusionSegmenter decision path runs end-to-end (no GPU/Gemini) on
88         EVERY
89         present golden trajectory and yields a handoff window list for each, for both gate
90         settings.
91         Asserts against the present corpus count (>= 6), not a stale fixed 6 ? the golden
92         set grows. """
93         results = sga.evaluate_manifest(_SPECS)
94         assert len(results) == len(_SPECS)
95         assert len(results) >= 6
96         for r in results:
97             assert r.num_gt > 0
98             assert r.n_handoff_off >= r.n_handoff_on # Gate A only ever drops windows,
99             never adds

```

```

95
96
97     @_skip
98     def test_served_gate_a_does_not_materially_regress_f1():
99         """SERVED no-material-regression guard: on the production windowing Gate A must not
MATERIALLY
100         hurt F1. On the present recall-starved multicourt corpus the served path is under-
segmented
101         (over-seg ratio < 1), so Gate A ? which only DROPS windows ? trims a few borderline-
true
102         windows and is mildly negative, NOT the ~neutral it was on the original single-court
6. We pin
103         a band that admits this measured mild loss but trips on a material regression."""
104         results = sga.evaluate_manifest(_SPECS)
105         agg = sga.aggregate(results)
106         # Honest measured value (2026-06-18, present 9-clip multicourt golden set): mean
served
107         # ?F1 ? -0.034 (was ? -0.008 on the retired original-6). Band re-measured to admit
the mild
108         # recall-starved loss while still tripping if Gate A materially destroys served F1.
109         assert -0.06 <= agg["mean_delta_f1"] <= 0.03, agg
110         # And Gate A must never collapse served F1 to ~half the OFF baseline (measured ratio
? 0.83).
111         assert agg["mean_f1_on"] >= 0.5 * agg["mean_f1_off"] - 1e-9, agg
112
113
114     @_skip
115     def test_served_gate_a_does_not_increase_over_segmentation():
116         """Gate A only ever DROPS windows, so the served over-seg ratio must not rise when
it's on
117         (the direction-of-effect invariant ? even where the absolute lift is ~neutral)."""
118         results = sga.evaluate_manifest(_SPECS)
119         agg = sga.aggregate(results)

```

22. assistant (2026-06-22T10:26:27.760Z)

Now I have a clear picture of the Phase-3 gap. The litmus uses ``os.path.isfile()`` to gate and plain file paths ? it skips entirely on a fresh box/CI and has no GCS-fetch path. Let me verify the train label-naming concern: with labels now named ``<date>_<name>.rallies.csv``, does ``name = fname.replace(".rallies.csv", "")`` yield ``<date>_<name>`` which then won't match the video stem ``<name>``?

The worker code at line 384: ``name = fname.replace(".rallies.csv", "").replace(".csv", "")`` ? for ``2026-06-14_GX010128.rallies.csv`` this gives ``2026-06-14_GX010128``. Then at line 387 it looks up ``video_by_stem.get(name)`` where video stems are ``GX010128`` (per Phase-2 plan ``videos/<name>.mp4``). This means **the date-prefixed label name will NOT match the date-free video stem** ? train with ``--trajectory-source detector`` would skip every video. This is a concrete bug introduced by the naming decision. Let me double-check the Phase-2 video naming intent.

The doc §2 Phase-2 says ``videos/<name>.mp4`` where "bucket name = the eval ``name``", consistent with ``trajectories/`` + ``labels/``. But labels are ``<date>_<name>`` while videos are ``<name>`` ? they are NOT consistent. This is the gap. Let me confirm trajectories naming too ? doc §7.2 says ``trajectories/<name>.csv`` (no date). So labels alone carry the date prefix, breaking the stem join in ``cmd_train``.

Let me quickly verify there's no stripping of the date prefix anywhere in train.

23. user (2026-06-22T10:26:29.732Z)

```

383:         fname = storage.parse_uri(uri).name                # e.g.
GX020094_proxy.rallies.csv
384:         name = fname.replace(".rallies.csv", "").replace(".csv", "")

```

24. assistant (2026-06-22T10:26:38.287Z)

Confirmed ? no date-prefix stripping. The label name `2026-06-14\_GX010128` will not match a video stem `GX010128`. This is a real correctness gap for `worker train --trajectory-source detector`, which is exactly the cloud-train path the doc points at in §4/§5 of `deploy/gcp/README`.

Note also: the GCS manifest (``manifests/golden.json``) carries entries keyed however the planner built them, but `worker train` does NOT read that manifest ? it derives `name` from the label filename. And the eval that DOES read the manifest (`served_gate_a`) can't fetch `gs://`. So the two consumers are split and neither is fully wired.

Let me quickly confirm whether the train code resolves the manifest naming when matching trajectories in cached source (it uses `synth/detector` only ? there's no `--trajectory-source cached``). The doc §5 gap table claims "train (`--trajectory-source cached``)" but the worker only has ``synth`` and ``detector`` choices (line 477). That's a doc-vs-code drift worth flagging.

I have enough concrete evidence. Let me compile the completeness gap list.

25. user (2026-06-22T10:27:43.233Z)

Structured output provided successfully

26. assistant (2026-06-22T10:27:54.112Z)

I've completed the completeness review and returned the structured verdict.

Headline: Phase-1 is genuinely done and consistent, but the goal "run ALL train+infer from the cloud, box no longer a blocker" is not reachable yet ? and two of the gaps are real code bugs the doc does not flag:

1. ****Blocker (code bug):**** the locked label naming ``labels/<date>_<name>.rallies.csv`` breaks the cloud train join. ``backend/worker/__main__.py:384`` derives the join key as the full filename stem (``2026-06-14_GX010128``) and matches it at ``:387`` against date-free video stems (``GX010128``), so the exact ``worker train --trajectory-source detector`` command in ``deploy/gcp/README.md`` §5 silently skips every clip and aborts.

2. ****Blocker (code, not config):**** Phase-3 "flip eval/litmus to GCS" can't be a config flip ? ``backend/eval/served_gate_a.py:110-111``, ``calibrate_wasb.load_golden_gts``, and ``tracknet_runner.parse_trajectory_csv:240`` all open paths with plain ``open()`` and the regression test gates on ``os.path.isfile``. No storage-layer fetch exists, so the ``gs://`` manifest is unusable and the litmus just skips on a fresh box/CI.

The remaining gaps (14 un-uploaded videos with no scheduled run-path, the 8 unproxied F: originals with no proxy tool, GPU quota still 0, per-VM secrets/creds + WASB weights re-staging, a ``--trajectory-source cached`` flag the doc cites but code lacks, and split train-vs-eval manifest consumers) are the org/infra checklist a next session will hit. Full detail, file:line evidence, and prioritized recommendations are in the structured output.